Exploring the viability of synthetic images to improve pothole detection under adverse conditions

Cian Spillane
Gaelcholaiste Carraig Uí Leighin
Stand 4209
Intermediate Technology
Individual

Comments page

Table of contents

Abstract **3**
Introduction **4**
Motivation 5
Objectives 6
Key Methods **6**
Machine Learning 6
Supervised learning 7
Unsupervised learning 8
Deep Learning 8
Neural Networks 9
Convolutional Neural Networks 10
Dilated Convolutional Neural Networks 11
EfficientNet 12
Generative Adversarial Networks 12
Residual Networks 13
Encoder-Decoder network 14
Image to Image translation models 15
Pix2Pix 15
Bicycle GAN 16
CycleGAN 17
UNIT 18
Object Detection 19
R-CNN 19
Fast R-CNN 20
Faster R-CNN 21
YOLO 21
SSD 22
EfficientNet 23
Experiments **23**
Data Augmentation 24
Data Collection 25
Training data 25
Validation data 26
Testing Data 26
Irish weather conditions 27
Synthetic Fog Implementation 28

Synthetic low light implementation..... 29
Synthetic Rain images implementation..... 31
New Datasets..... 34
Deep learning model..... 35
Object Detection choice.....35
YOLO v7 size experiment.....36
Transfer Learning..... 37
Training.....37
Results..... 40
Experiments.....40
Evaluation metrics..... 40
Recall.....40
Precision.....41
mAP 0.5.....41
Speed..... 41
Results.....41
YOLO v7 size results.....41
Baseline vs Synthetic vs Hybrid..... 43
Baseline results..... 43
Synthetic results..... 45
Hybrid results.....47
Discussion..... 49
Conclusion.....50
Real world applications.....51
Appendices..... 52
Appendix A - Real world pothole detection.....52
Appendix B - YOLOv7-d6 code..... 57
Appendix C - Synthetic Images..... 61
References..... 81
Acknowledgements.....84

Abstract

Potholes sometimes known as pavement defects pose a significant problem for the Irish transportation industry. Potholes not only put your vehicle in risk but they can also pose hazards to your life if deep enough. In Ireland alone a survey from the AA revealed that a staggering 60% of Irish individuals have had their vehicle damaged in some shape or form from potholes in this country[1]. In 2023 Ireland for the third year running spent 56 million euros to try and improve the

issues we Irish citizens face with potholes every day[3]. With the hazards posed by potholes and the inordinate costs associated with repairing them it is crucial to detect potholes before they can expand even further.

Artificial Intelligence(AI) is one field that has shown significant potential to improve the safety of Irish citizens through detecting potholes. If the potholes are detected at an early stage the local authorities can solve the issue before it expands into a more hazardous and expensive issue. Despite the developments in AI in recent years there is still uncertainty over just how reliable these models really are. A recent paper demonstrated the drastic decrease in accuracy that object detection models experience when tested on data under adverse conditions. The issue with most of the current methods of pothole detection is that they are trained solely on images of potholes under clear and sunny conditions. These models are also tested under similar conditions which leads to a deceivingly impressive performance. However these models don't include potholes under adverse conditions in their training data such as potholes under rain and fog. This underrepresentation of potholes under diverse conditions leads to worse performance in adverse conditions.

Biases in training data lead to less generalizable models and also puts certain countries, areas and regions in a significant disadvantage[4]. Countries with more diverse climates including more snow, fog, rain and darker and damper conditions are the victims of this decreased performance. Countries, areas and regions then with sunnier and more clear climates are at an advantage. Current methods of pothole detection for the most part are not both generalizable and accurate enough under different conditions to be deployed at scale. Various approaches have been developed over the years to try and tackle this issue all offering new insights and advantages[4][5].

My project steps in to solve this issue. My work looks to directly solve these issues through using image to image translation models to generate synthetic data.mimicking adverse conditions such as rain and fog. With these synthetic images I can create a dataset that is both larger and has a more diverse representation of different weather and lighting conditions.

My project proposes a CNN based object detection model which can be used to detect potholes in real time. My model achieves this by utilizing state-of-the-art image to image translation models and vigorous experimentation. My final experiment demonstrated that synthetic images drastically improve the detection accuracy of potholes, with a staggering 8.6% accuracy increase in accuracy when tested on damp/wet potholes.

Introduction

Pothole detection is an important task, ensuring there are no hazardous potholes on the road protects the 2.9 million licensed vehicles in Ireland[8]. Potholes occur from environmental conditions. The process begins with traffic which forms small tears in the road. Water then seeps into these small tears and as the water freezes the small tear expands forming a larger void in the road.[9] AI pothole detectors can be a useful aid for manual road inspectors. With AI pothole

detectors the inspector can simply drive over a road and the AI model will identify the relevant regions containing potholes. If a pothole is detected early enough before expanding the local councils will then be able to repair it at a fraction of the cost of repairing it if it expands. Through harnessing AI pothole detection model's I believe we can not only ensure the safety of Irish citizens but we can also greatly decrease the government and country council's expenditure on pothole repairment.

Motivation

Potholes are not a new issue and have been a constant issue on roads all over the world for numerous years. The current approaches to pothole detection show little change from the traditional methods. The current method involves paying a team of road inspectors to physically examine the condition of roads and detect any potholes manually. Not only is this an incredibly time consuming task it is also extremely expensive. However it does look like a new method of pothole detection will be implemented soon that being pothole detection using AI. It is estimated that by using AI as an aid for pothole detection the expenditure on pothole detection will decrease by 50%.[10]

.

Potholes are a major inconvenience for both Irish citizens and the local road authorities in the Country. The issue of potholes isn't just a major issue in Ireland, it's a major issue worldwide. In San Diego alone it has been reported that over 30, 000 potholes are repaired each year to improve the maintenance of their road infrastructure. Although this likely amounts to a very high cost it is actually a lot more effective than what most areas do with potholes. San Diego prioritizes fixing potholes in their early stages, as mentioned earlier this greatly reduces the costs associated with pothole repairment.

Nearly every country suffers from potholes to an extent. In 2017 New Zealand reportedly spent 525, 000 (New Zealand Dollar) on pothole detection and repair in Christchurch alone[12]. Although AI tools offer the potential to greatly reduce these figures it's still incredibly tricky to find pothole data under diverse conditions.There are no publicly available images of potholes with rain or fog in the background. Current methods of pothole detection underrepresent certain lighting and weather conditions which leads to decreased performance on those conditions. Ireland would have an extremely mixed climate meaning we would be at a disadvantage here too.

Current research on pothole detection models are limited and the models developed are simply not generalized enough to be used by manual road inspectors. These models struggle to adapt to new weather, lighting and image types. After my BTYSE project in 2023 which explored bias in machine learning I was motivated to tackle this issue. Tackling biases in object detection in general is an important task not just for infrastructure maintenance but it has applications towards creating safer self-autonomous vehicles[13].

Objectives

The main objective of my project is to apply deep learning techniques to existing pothole data to create a more diverse dataset. This will then be used to train a more generalized pothole detector that can generalize to new conditions more effectively than current methods. My objectives for this project can be summarized as the following:

- Study the issue Ireland faces with potholes
- Provide a comprehensive analysis of the current models used for pothole detection
- Apply image to image translation models to existing pothole data to create a more diverse dataset of potholes.
- Train a CNN based object detection model how to detect potholes in images or videos.
- Ensure my pothole detector is more generalized than current options.
- Propose a new model, including multiple new improvements so we have a pothole detector that is better suited to Ireland's unique climate than current options.

Key Methods

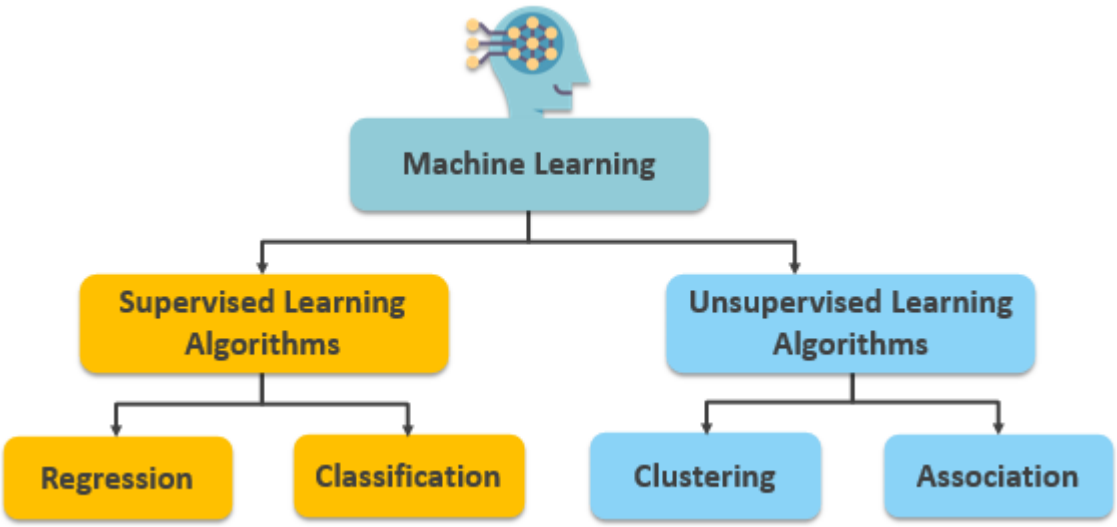
The following provides brief explanations for key methods used in both image to image translation and object detection. It also provides explanations for the developments over the years in image to image translation and object detection.

Machine Learning

Machine Learning is a subset of AI that is based on using mathematical and statistical models to make predictions[15]. The term ‘Machine Learning’ was first introduced by Arthur Samuel[14], Samuel at the time was working at IBM and was responsible for one of the first self learning-programs. Samuel created a checkers playing program which would gradually improve each time it played. This work laid some of the earliest foundations for machine learning and AI in general.

Machine learning is a subset of AI that can be described as the scientific study of using mathematical methods to solve problems and make predictions. The key idea that has led to so much attention.in recent years is that machine learning models gradually improve. The model’s learn from their mistakes and slightly tweak their techniques and parameters to try and improve their performance. Machine learning can be also defined as the study of training models to make predictions without explicitly programming them. This means that each model learns patterns in it’s training data to then make predictions on new unseen data.

Machine learning is a broad field that can be split into many more sub sections. Two important sub sections in machine learning are supervised and unsupervised learning.



Supervised learning

Supervised learning is the most popular method of learning in machine learning.Supervised learning is when a model learns from being given a set of input data(x) and its corresponding answers or labels(y). The model learns the mapping between the input data(x) and its corresponding label(y)[16].

The model has to start with parameters which can be described as P. They are essentially the value that determines the behaviour of a machine learning model. It usually starts with a random number and as the model learns during training seeing each new piece of data it updates its parameters to slightly improve its predictions.

The goal is to teach a function how to accurately predict the output label(Y) from the input label(X). This is often represented as:

$$Y=F(X)+E$$

E is equal to the error of the model which is the difference between its prediction for Y and the actual Y. The goal is to try and minimize the error in the function.

Supervised learning can also be divided into two categories in regression and classification. Classification refers to classifying something into specific classes while regression means making predictions with an infinite number of possible outputs. Some popular supervised learning algorithms include: support vector machine(SVM), linear regression and logistic regression.

Unsupervised learning

Another type of learning in machine learning is unsupervised learning. Unlike supervised learning, unsupervised learning is trained solely on input features(x) and is given no corresponding correct labels(y). What's interesting about unsupervised learning is it receives no feedback from its environment; it's just given input features and is expected to make predictions from that[17].

Unsupervised learning models have to find structures and patterns inside its input data on its own. The one major advantage of using unsupervised learning and that's that no labels are needed. Labeling large amounts of data takes a long time and is manually tiring[18]. With unsupervised learning the models just need the input data and can find its own patterns.

The most popular use of unsupervised learning is clustering. Clustering is putting a group of data into clusters based on their data points. For example google search utilizes clustering algorithms. If you search 'Next Irish President' it will find all the articles containing the words 'Next', 'Irish', 'President' and will sort them from top to bottom based on popularity.

The three main unsupervised learning algorithms are clustering, Association Rule Learning and Dimensionality Reduction.Unsupervised learning is still relatively undeveloped in comparison to supervised learning.

Deep Learning

Deep Learning is a sub-field of machine learning. Deep learning takes its inspirations from the human brain and looks into how we can utilize neural networks to solve complex problems. Deep learning is typically considered more powerful than machine learning and in recent years has been the main method of AI based pothole detection. Deep learning is a rapidly developing field with a lot of it's core ideas based on the method Geoff Hinton introduced in 1988 called backpropagation[19].

The following sub sections briefly explain the main architectures in deep learning that have been used in pothole detection and image to image translation over the years.

Neural Networks

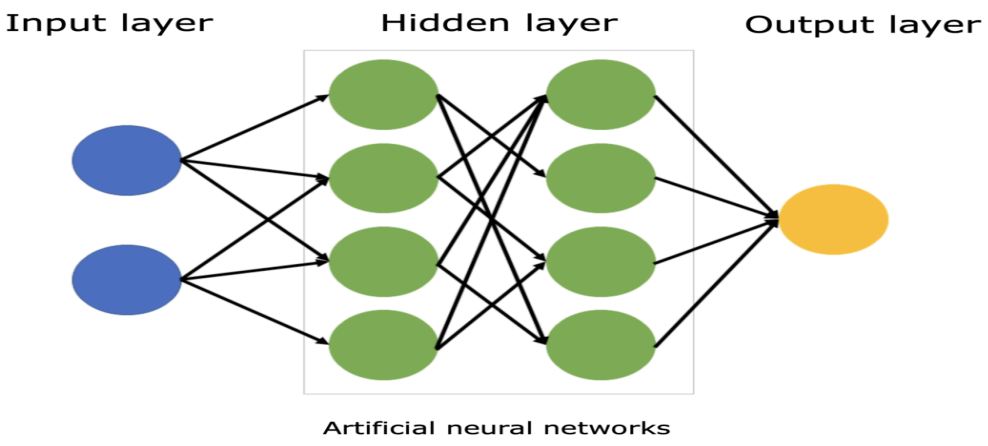
The Neural Network is an abstraction of the human brain. Neural networks consist of artificial neurons based on the way that the human brain is composed of neurons. Each neuron has inputs and produces a single output which can then propagate forward to the next neuron.

The output of a particular neuron can be worked out as the following:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Where:
Z = output of the neuron
n = number of inputs
w = weight of connection between neurons
x = input values
B = bias term

The value of a neuron is the sum of its inputs(x), multiplied by the weight of its connections(w) plus a bias term(b). This weighted sum is then passed through a non-linear activation function to produce the output value. Each value is passed on to the next layer until it reaches the final layer where the model makes a prediction.



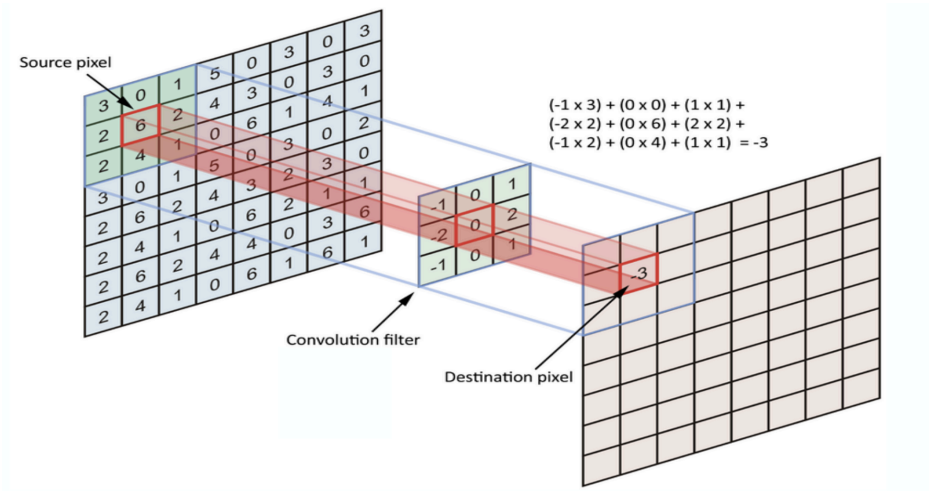
Once the model makes a prediction the model's prediction is compared to the correct answer. The difference between both is called the loss function. Learning involves propagating backwards to change the value of the weights to minimize the cost function. Backpropagation calculates the gradient of the cost function with respect to the weights and biases. The weights are constantly updated to minimize the cost function and optimize the performance of the model using various methods, some common include and are not limited to Stochastic Gradient Descent(SGD), Adam and Momentum. Neural Networks are a powerful tool that is extremely important to pothole detection.

Convolutional Neural Networks

Convolutional Neural Networks(CNN) are a type of neural network that is mostly applied to image classification. The main difference between the baseline neural network of fully connected neural networks and the convolutional neural network is that CNNs are less sensitive to small changes in input making them more suitable to work with images.

Just like fully connected neural networks, CNNs are inspired from the human brain and compose of multiple connected neurons. Although the origins of CNNs trace back as far as 1980 when Kunihiko Funushima initially introduced the idea[20], Yann LeCun(META AI leader) is the man who applied backpropagation first to CNNs and is often seen as the father of CNNs. LeCun achieved impressive performance with this as he trained a model that could classify different handwritten digits. As far as I know this is the earliest use of CNNs[21].

The architecture of CNNs is slightly different to fully connected neural networks. CNNs first take the input image and apply convolutions to that image. Convolutions are used to detect edges and patterns. Learnable filters often known as kernels are applied to this image. These filters slide over the image taking a small area of the image and turning it into a matrix. The values in the filter matrix are then multiplied by the values of the input image to calculate the dot product of that area of the image. Each value is then moved to a feature map.



A bias number is added to each number and when the feature map is completed it is put through(not always but usually) a non-linear activation function to produce an output.

Another new operation is then applied called max-pooling. Max pooling selects the maximum value feature in each region of the feature map to make the feature map smaller. Max-pooling reduces the size of the feature map while retaining key information, this is done so the model is computationally quicker. For example a feature map of a size 4 x 4 could be flattened into a 2 x 2 feature map with the key features using max pooling.

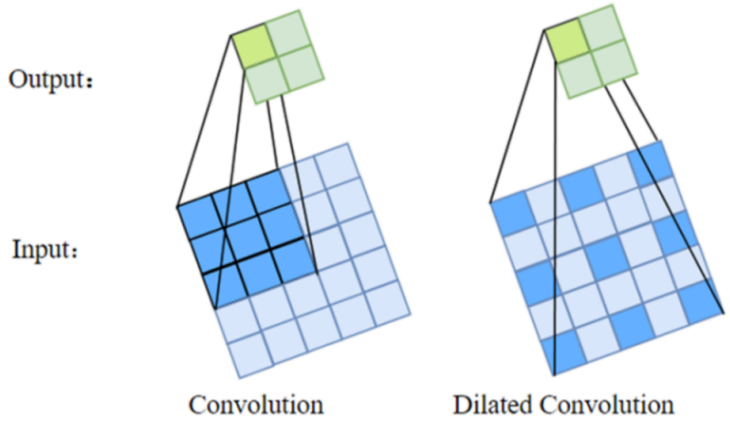
Then we use the remaining values from the feature map as nodes for the neural network. These nodes are just like fully connected neural networks described above multiplying the value of the node by the weights of its connections + a bias term. This value is then put through a non-linear activation function to produce an output. Backpropagation is then employed to minimize the cost function and optimize model performance.

CNNs are a widely used algorithm in pothole detection and have seen some very impressive results over the years. Several state of the art models discussed later in this paper are built on top of the foundations of this technique[22].

Dilated Convolutional Neural Networks

Dilated convolutional neural networks (dilated CNNs) is another type of CNN that has been applied to pothole detection. Dilated CNNs were introduced by Fisher Yu and Vladen Koltun[23]They propose small changes to initial CNN architecture. Dilated CNNs are the exact same as the regular CNN described above, the only difference is that dilated CNNs use dilated convolutions as opposed to CNNs which use the sliding kernel method based convolution.

A dilated convolution has a larger receptive field than a regular convolution giving the network a larger view of the image without using any extra computation. The way this is done is by instead of sliding over the input image and taking a small matrix exactly from that image and then multiplying it by a kernel dilated convolutions use the same area as normal convolutions however they use gaps so they can fit more of the image into their matrix.



Dilated convolutions use less convolutions than CNNs do which saves space and storage. The rest of the architecture is the exact same as a CNN with the feature maps generated from the dilated CNN we can then use max-pooling to make it smaller and then put the remaining values through a fully connected neural network to generate the output and then apply backpropagation. Dilated CNNs have been applied to pothole detection and have shown some interesting results[24].

EfficientNet

EfficientNet is another type of CNN short for Efficient Network. EfficientNet was introduced by Mingxing Tan and Quoc V Le in 2019,[25] with the idea of making a more efficient model that uses less computational resources than others while still achieving state-of-the-art performance.

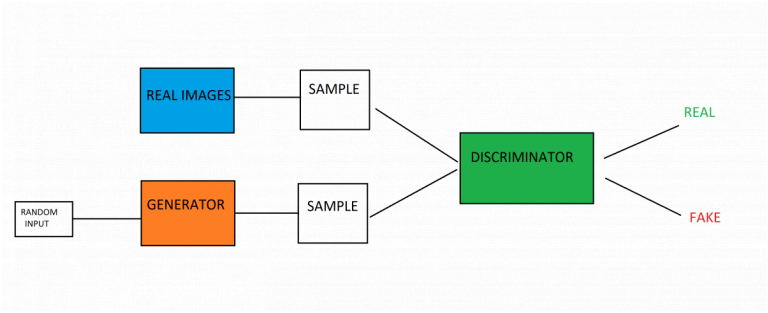
The key idea from EfficientNet is that we usually scale up our models wrong. When we try to scale up the model's width(adding more feature maps)), depth(deeper networks with more layers) and resolution(higher image resolutions). Scaling these up leads to an accuracy improvement to an extent but is computationally a lot more expensive. EfficientNet proposes compound scaling. This means we scale the model's depth, width and resolution simultaneously with a balance. This balance means all three factors are scaled very similarly which this paper finds to be very effective.

The EfficientNet architecture uses its own specific activation functions and convolutions to save resources. EfficientNet introduces a simple yet very important concept in 'compound scaling' which is used in various top models in both image classification and object detection today.

Generative Adversarial Networks

The GAN architecture was initially introduced by Ian Goodfellow in 2014[26]. GAN's consist of two models: a 'generator' and a 'discriminator'. These two models play a game to see who can fool who and they both gradually improve by playing this game.

Assuming we want to train a GAN to generate ultra-realistic synthetic potholes the generator will start with random data and will create a random image which will look nothing like a pothole. The Discriminator then, which is trained on images of potholes, will then say whether it thinks what the generator has generated is a pothole or not. The Discriminator gives feedback and through this feedback the generator gradually improves at generating potholes to a point where it is capable of generating images that are not actually potholes but the discriminator thinks are. The discriminator is trained to differentiate between real and generated potholes, meaning the generator is not trained on anything. The generator learns from its feedback from the discriminator. Both models consistently test each other which leads to both model's improving.



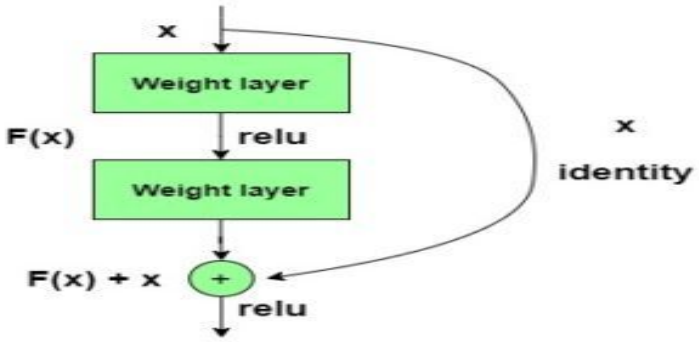
GAN's are a very powerful and exciting architecture that are the engine behind deep fakes. That being said GAN's can be used in a positive manner. Later in this paper we discuss image to image translation models built on top of this architecture

Residual Networks

Residual networks are an important method that has applications to both pothole detection and image to image translation(creating synthetic images). The Residual Network better known as ResNet architecture was first proposed in 2015[27]. The goal of the research was to find a solution to the vanishing gradient issue. As we train extremely deep and powerful neural networks with loads of layers in AI the gradient tends to explode leading to poor model performance. The gradient shows us how different weights change the loss function, however with extremely deep models the accuracy degrades quickly.

To solve this Kaiming He at el introduced the Residual Network Architecture in 2015. Residual Networks solve this issue by allowing models to use residual connections or sometimes known as

skip connections. These connections allow the model to learn mapping between the input and output instead of the original mapping.



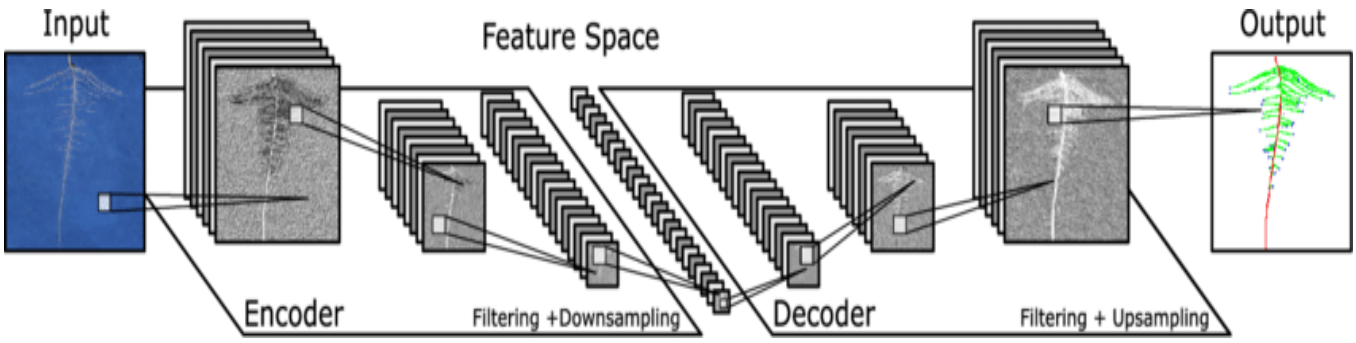
The original mappings refer to the operations that the network goes through like convolutions, max pooling and fully connected layers. Residual mappings instead re-apply the input with the output of these operations so through residual learning the model can learn the difference(the residual) between the input and the output as opposed to the traditional method of going through all the operations required. This allows us to train extremely deep models without running into the vanishing gradient issue. ResNets allow use to train deep neural networks for pothole detection and are commonly used in most object detection models.

Encoder-Decoder network

The encoder-decoder network was initially developed for text generation and text translation but in recent years has been applied to image generation. The encoder-decoder network was introduced by Ilya Sutskever, Oriol Vinyals and Quoc V Le in 2014[28]. Their work introduced the idea of using two models, an encoder and a decoder and this same structure is used in some of AIs most powerful models such as large language models and image generators.

The idea is to have two networks: an encoder and a decoder. The decoder is trained how to translate something. For example if we want to translate a noisy blurry image into a clean image with no blur or more simply remove the blur from an image. The model is trained on images with and without blur. The encoder takes the input a blurry image, applies convolutional operations to extract important features and compresses it into a smaller representation of the image(sometimes known as a feature map) in the form of a feature vector.

The decoder first translates the this feature vector into its corresponding feature vector from the mappings it has learned between the input and output.The decoder takes this vector(compressed representation of the input image) and applies deconvolutions(often known as up-sampling) to expand the image back to the original image size. The output image is the same as input image but without the blur and the encoder decoder network has learned the mapping between the two. The encoder creates a compressed image without the blur while the decoder uses this and deconvolutions to expand the image and create the output[29]. Residual networks are used to learn the mapping between the input images and output images.



It's worth noting that the encoder decoder is slightly different when working with different forms of data. If it was working with text it wouldn't use convolutions but instead it would use another network such as recurrent neural networks. The encoder decoder architecture is a key part of how image to image translation works.

Image to Image translation models

Image to image translation aims to transfer images from a source domain to a target domain while keeping the content representations[30]. This just means you're taking a picture and translating that picture into the exact same picture but with a different domain. The model still keeps the same structure as the original image. Image to image translation is a very new field that has garnered some interesting results. My idea is to use image to image translation to generate synthetic images of potholes under different weather and lighting conditions topical to the Irish climate. This means I take an image of a real pothole and use a model to translate this image into the exact same image but under a different condition like rain. The following includes some of the key models and developments in this field over the years and the current state-of-the-art options.

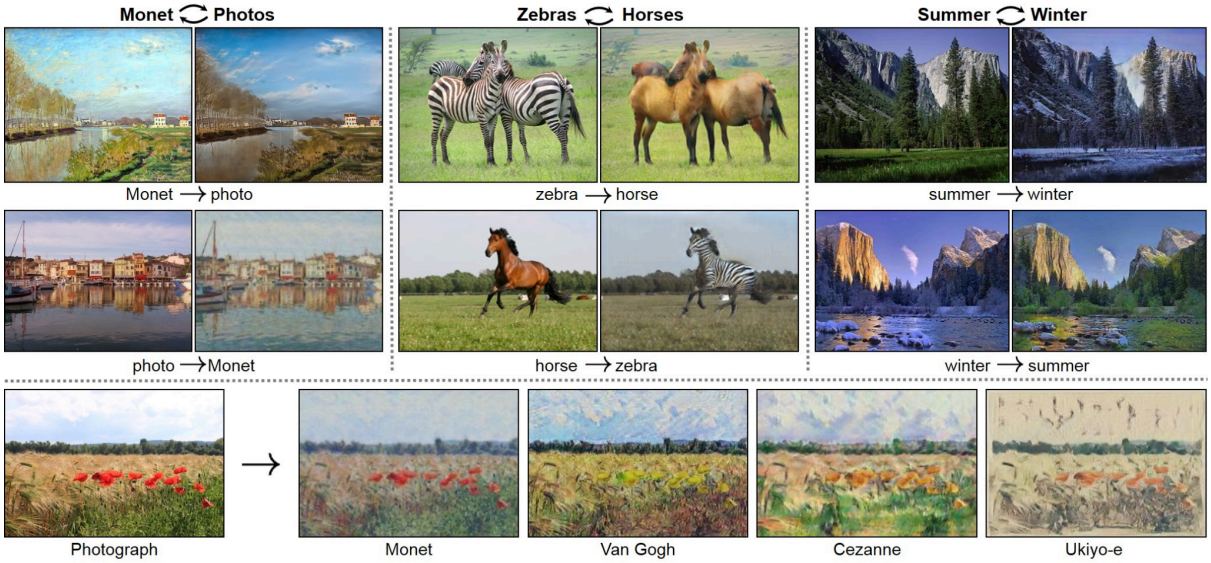
Pix2Pix

Pix2Pix was the first major breakthrough in the field of supervised or paired image to image translation. Pix2Pix introduced in 2016 took the machine learning industry by shock in achieving outstanding results[31]. Pix2Pix is known as the first image to image translator to achieve such results.

What Pix2Pix introduced that hadn't been used before is conditional generative adversarial networks. The idea behind this is similar to a regular GAN except it adds a condition to both the generator and discriminator. This GAN is trained on paired/supervised data. The model receives the input image and desired output translation. With this the model can learn the mapping between the two. Conditional GANs work similarly to GAN except the generator instead of just generating random images and being given feedback it is given a condition while the discriminator instead of

just judging if it's real or not it checks if it is realistic or not and if it fits the condition. Adding this condition is powerful and in the use of translating potholes from clear to rain the condition would be something along the lines of ‘generate a rainy image’. The discriminator then checks this condition and judges whether the image is rainy or not.

Pix2Pix takes paired data meaning it learns in a supervised manner. It uses residual networks to learn the mapping between the input image and its corresponding output. The pix2pix architecture consists of a generator and a discriminator. The generator uses the encoder decoder network to downsample the input image into a feature vector. This vector is then translated into its output feature vector and the decoder it used to upsample to feature vector into its original image size. The output of this is the exact same image as your input version but with a new domain present in the image. The discriminator is then used to judge whether the image is realistic or not and both models: the generator and the discriminator gradually improve.



Bicycle GAN

Bicycle GAN is another supervised or paired image to image translation model that proposes a small change. Bicycle GAN was introduced in 2018 by 7 authors[32]. Pix2Pix despite its results can run into a formidable issue which is called ‘mode collapse’. This issue happens when in the training paired training data more than one input is paired with the same output. This happens when the generator generates images with little diversity due to unbalanced training data.

To solve this Bicycle GAN applies a bijective which means every training image input and output are mapped together one on one ensuring no inputs map to the wrong output. This is done through combining two networks called cVAE-GAN and cLR - GAN. These networks essentially enforce a stricter one on one mapping between all the training images. This bijective is a simple addition but can be helpful when you run into issues with matching training data when working with image to

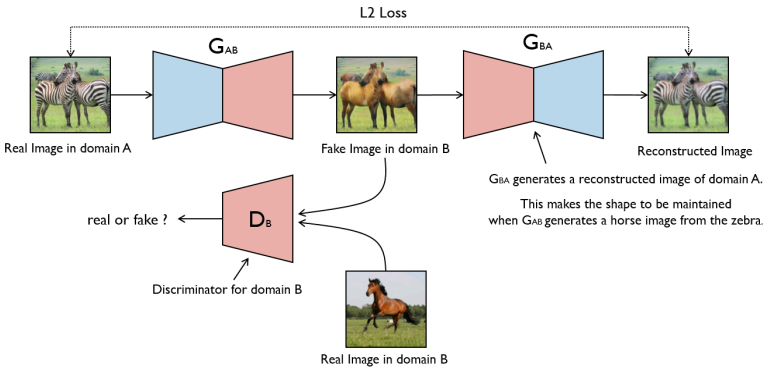
image translator. Just like Pix2Pix this works essentially the same and is known as a supervised image to image translator.

CycleGAN

CycleGAN, short for Cycle Generative Adversarial Network, introduces cycle consistency into its architecture for the first time[33]. The paper was introduced in 2017 by Jun-Yan Zhu, Taesung Park, Phillip Isola and Alexei A. Efros. Paired data isn't available for every task and CycleGAN addresses this by developing an unsupervised image to image translator.

Cycle GAN uses 2 generators and 2 discriminators. The model is trained on two separate image domains and learns the mapping between these two although neither of them have corresponding labels. CycleGAN learns the structure and patterns between both image sets on its own.

The first generator(G) takes the input image and uses the encoder decoder network to encode the image and apply convolutions to extract the key features and style of the image into a smaller representation. Just like Pix2Pix this small representation of the image often known as the feature map is translated into the same feature map but with the targeted domain's features. The decoder performs upsampling to expand the image back to its original size. The model learns patterns from the data provided and understands through the condition the task of translation required. The discriminator then judges whether this generated image is realistic and whether it fits the condition(domain). This discriminator gives feedback so the generator improves.



The second generator introduces 'cycle consistency' , the technique that enables effective translation and performance even without paired images. The now translated image which has been translated from the input to its output domain is passed to a second generator(F) and put through another encoder decoder network. This network encodes the image into a smaller representation capturing the key style and features and translates it back to the original input domain. This introduces cycle consistency. The decoder uses upsampling to expand the image back into full size while a second discriminator is used to compare this reconstructed image to the original input image. This reversed image which is supposed to be as close as possible to the input

image is often called the ‘reconstruction’. This technique helps the first generator(G) learn the mapping from winter to summer and the model is expected to return nearly the same image as the original image if done correctly. If the reconstruction is very different to the original image then the discriminator gives the generator feedback to improve its translation. This cycle consistency technique helps the model learn the mapping between input and output better and allows the generator to improve its performance.

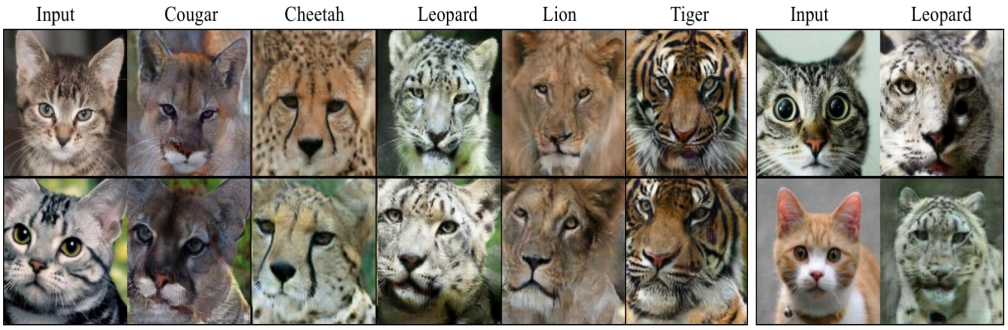
CycleGAN was used during my project as I wasn’t working with paired data and essentially what CycleGAN proposes is the use of cycle consistency through a second generator and discriminator to introduce cycle consistency.

UNIT

Unsupervised Image to Image Translation best known as UNIT is another popular state-of-the-art unpaired image to image translation model. This model was released in 2017 by NVIDIA researchers with one key difference that builds on top of CycleGAN[34].

The UNIT paper introduces the idea of ‘shared latent space’. Latent space refers to the small representation of the input or output image that includes the images key features and styles. This latent space is used to create a small representation of the image which can then be translated into the desired domain. In Cycle GAN the idea of cycle consistency is introduced. This applies another generator and discriminator to translate the desired domain backwards to test the effectiveness of the model’s performance. If the generated reconstructed image is very similar to the original image then the model is working well. However in Cycle GAN both generators use different latent spaces. The first generator which takes the input domain and encodes it into a latent representation of that input which is then translated into the output domain representation in the decoder. Then in the second generator the output desired domain is encoded into a different latent space. The reason the latent spaces are different is because there is a different domain in the input and output image which affects the latent space.

UNIT proposes the idea of using the same latent space twice for both generators. The idea is that although the model will miss out on some content the model will learn a deeper mapping between the input and output domains. UNIT works exactly like Cycle GAN in the sense that the 1st generator is used to create a latent space of the input image and then the decoder translates this image into the desired domain and uses deconvolutions to expand the image back to its original size. The discriminator then judges these images The difference between Cycle GAN and UNIT lies in the cycle consistency and the second generator. Instead of encoding the output image into a new latent space we use the same latent space encoded from the input image again. This is then decoded and expanded back to the original image.



UNIT proposes the use of a shared latent space which leads to a more generalized model. The image above shows a use case of UNIT translating an input cat into other animals while maintaining the core structure of the cat.

Object Detection

After debate and research which is described in detail later in this paper object detection was chosen as the computer vision based method to detect the potholes. Through using image to image translation we can generate a more diverse pothole dataset that can then be used to train a state-of-the-art object detection model using transfer learning.

Object Detection is the task of both detecting and localizing an object in an image or video. Object detection is built on top of the CNN method defined earlier. There have been hundreds of object detectors built over the years so it was impractical to analyze them all. The following provides brief explanations of the main object detection models.

R-CNN

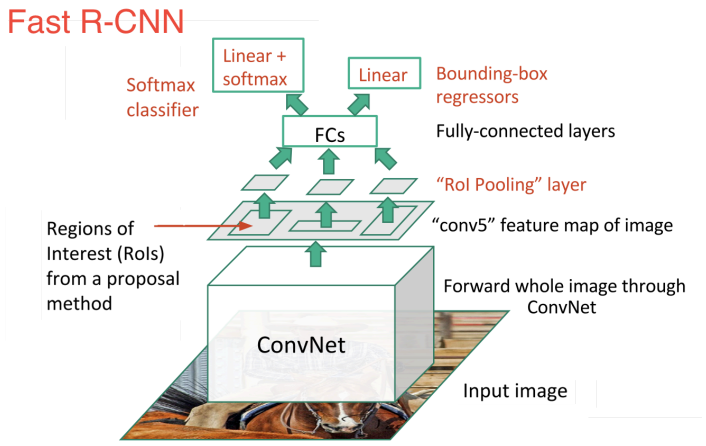
The R-CNN stands for Region Convolutional Neural Network.The model was first introduced by Ross Girshick, Jeff Donahue, Trevor Darrell and Jitendra Malik but has seen significant improvements from different authors over the years[35]. This architecture combines regional proposals with a CNN. The typical method of object detection before R-CNN was either using traditional machine learning techniques or hand crafted feature detectors. R-CNN applied region proposals through a method called ‘selective search’. This method takes in the input image and generates about 2000 regions from that image. These 2000 regions are then passed through a CNN that uses convolutions and max-pooling to detect the key visual-features of each region. These regions are then extracted as a feature vector(sometimes referred to as a 1d tensor, the model takes in a 3d tensor and extracts a 1d tensor).

A supervised learning classification algorithm called ‘Support Vector Machine’ often named SVM is trained on a large dataset of images with their corresponding annotations. These annotations include the bounding box for the object of interest in the image. The SVM then scores each

extracted region. Sometimes we run into the issue of many regions being proposed that have the same object in the image. To solve this ‘non-max suppression’ is used which rejects a region with a lower intersection over union with another region. R-CNN is a relatively simple detector that is known as a two stage detector meaning its first stage is the region proposal and the second stage is the CN and classification. The R-CNN has some obvious disadvantages, one of the most notable is how slow the region proposal stage is making the model very inefficient.

Fast R-CNN

Fast R-CNN standing for fast region convolutional neural network as its name may suggest is a faster version or the aforementioned R-CNN. The idea was proposed in 2015 by Ross Girshick to develop an object detection model similar to the R-CNN model however a lot faster and a lot more efficient[36]. It does this in 2 ways: it applies an extra region of interest(ROI) pooling layer and also the idea of using the same CNN for each region proposed by selective search instead of a different CNN for the 2000 different regions.

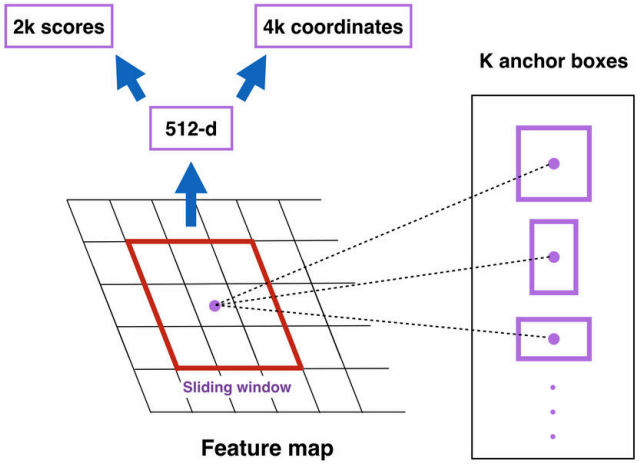


Fast R-CNN uses selective search to generate 2000 regions in the input image. These 2000 regions in an R-CNN would 1 by one be put through a CNN making it an extremely slow operation, Instead fast R-CNN puts the entire input image with the 2000 regions on it through 1 CNN together. The CNN applies convolutions and max-pooling to generate a shared feature map containing all the proposed regions. The new method ROI or region of interest is now used to take the shared feature map with all the regions provided and resizes all the regions in the image to a fixed-sized vector. With all the relevant information in one image the image is put through a softmax classifier to classify the class or classes in the input image. A separate classifier is used for the bounding boxes. Fast R-CNN provides a significant increase in speed compared to R-CNN while maintaining similar detection and localization accuracy.

Faster R-CNN

The faster R-CNN is another extension of the R-CNN family that provides even more improvements in terms of speed and efficiency compared to the previous models[37]. The one main issue with Fast R-CNN is the selective search algorithm used to generate areas in the image. This algorithm is an external algorithm which isn't actually built for object detection meaning it doesn't run in real time. This slows down the model quite a lot.

Faster R-CNN solves this by replacing the selective search algorithm with Region Proposal Networks(RPN). The difference here is that instead of instantly generating regions like R-CNN and fast R-CNN in Faster R-CNN we first take the input image and put it through a CNN. The CNN returns a feature map which then we apply to the RPN. The RPN is part of the CNN algorithm meaning it works straight away in real time. The RPN takes the feature map and slides a kernel(filter) over the feature map to propose anchors. Anchors are predefined boxes in the image. Each anchor is then analyzed by the RPN to work out if it is a foreground(contains something) or background and work out the 4 coordinates for each image.



A ROI(Region of interest) is then applied to the feature map generated by the RPN which includes all the regions that are foregrounds and their coordinates. The ROI resizes these to a fixed-sized vector and these re-sized feature maps are put through a classifier to classify the class of each region and a separate classifier to work out the coordinates.

In conclusion, faster R-CNN is the same as R-CNN except it replaces the external selective search algorithm with internal Region Proposal Networks which are faster than the traditional Selective Search algorithm.

YOLO

YOLO standing for You Only Look Once is the most widely used object detection model in the world. YOLO is best known for its balance between speed and accuracy, finding this balance is

very important as although some models can achieve state-of-the-art accuracy they are often too slow or use too many separate models to make the model too slow and expensive to be deployed at real time.

YOLO in 2016 proposed a one-stage detector[38]. The other object detectors we have analyzed to date have two stages: the region proposal stage and the classification stage; however YOLO handles region proposal and classification within the same CNN together. The idea behind YOLO's method was humans only need to look once and they know what is in front of them so why don't object detectors work this way too?

According to the paper the YOLO architecture consists of 24 convolutional layers and 2 fully connected layers. The model attempts to predict the class of the objects in an input image, the coordinates(bounding boxes) for that image and provides its confidence all in one look. The model is trained on the same data as other object detectors(supervised). It starts by processing the input image and putting it through the convolutional layers to generate the feature maps. The feature maps are split into S x S grids or 7 x 7 grids. Each grid is responsible for identifying the class of the object inside its grid, the coordinates of that object and its confidence score in its own grid. The feature map with the grids is flattened into a 1d vector which is put through a fully connected layer which predicts the class, coordinates and confidence for each grid. These are then combined to calculate the overall scores.

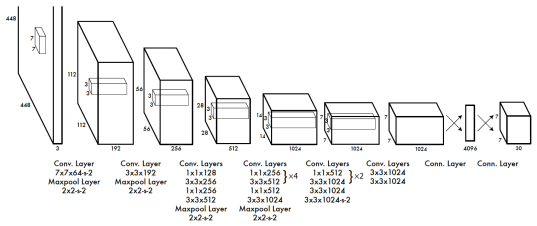


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

YOLO achieves a very impressive 45 FPS(frames per second - an important evaluation metric for speed of object detectors). This is extremely impressive as the next fastest detector we have seen so far in Faster R-CNN detects 7 FPS. According to the papers of Faster R-CNN and YOLO the mAP(Mean average precision - main evaluation metric for accuracy of object detectors) of YOLOv1 is just 4.2 percent lower than Faster R-CNN while being significantly faster. YOLO utilizes a powerful architecture to provide the ideal balance between speed and accuracy making it an extremely popular model. The YOLO family consists of 11 models all offering slight improvements and new features.

SSD

SSD standing for Single Shot Multibox Detector was also proposed in 2016 by Google[39]. SSD is another type of single shot object detector meaning it's very fast and efficient. The idea behind

SSD was to build a slightly more accurate model than YOLO but as a result it would be slightly slower. When tested on the VOC207 dataset SSD shows very impressive results.

SSD proposes 2 main small changes to the YOLO architecture, the first is it uses multiple feature maps for different sizes. YOLO goes through convolutions to generate a feature map. The feature map is flattened and put through FCNs to predict. The feature map is split into grids to calculate the class, coordinates and confidence of each grid. SSD uses multiple feature maps generated by different layers to handle objects of various sizes. Another addition is the addition of the anchor method spoken about is faster R-CNN. The main reason for this was to improve performance when more than one object was in a cell.

These two small changes offered advantages in comparison to other object detection models. It achieved a very similar mAP score to faster R-CNN when tested despite being only a one shot detector.

EfficientNet

EfficientDet stands for efficient detection and is based on the aforementioned EfficientNet network. As its name may suggest EfficientDet prioritizes the balance between efficiency and accuracy like YOLO and SSD. EfficientDet is also a one stage detector. This model was proposed by the Google brain research team and has gathered a lot of attention due to it’s impressive performance[40]

EfficientDet takes the input image and uses its own EfficentNet network to apply convolutions and generate multiple feature maps. EfficentNet just like SSD proposes multiple feature maps of different sizes like different resolution, width and depth. These feature maps have diverse depth, width and image resolution. The model uses anchor boxes like SSD also to split up the image.

EfficientNet released slightly after the initial paper a method called BiFPN(Bi-directional Feature Pyramid Network. Essentially what this BiFPN does is it fuses different feature maps together. It will fuse the different feature maps experimenting and eventually outputs one feature map with the optimal scales. This feature map is then sent through the fully connected layers to work out the class and coordinates in the input image.

EfficientDet is an extremely popular object detector that is typically known as the most accurate single shot object detector while losing much speed

Experiments

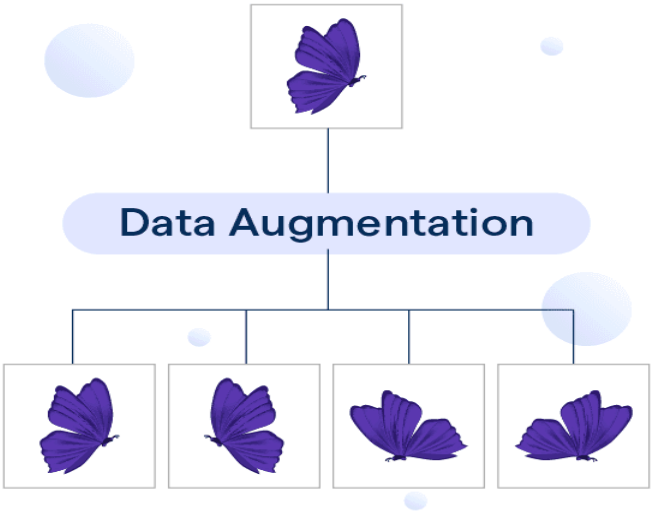
Using my datasets described later I implemented 3 image to image translation models to perform image to image translation. These conditions I translated to were specifically designed to be applicable to the Irish climate. Once the additional synthetic datasets were ready I could begin my final experiments.

The goal of this work was to study and analyze the current methods of pothole detection and propose my own novel improvements to eventually train my own novel pothole detector better suited to Ireland's diverse climate. My goal is to create a more generalized model using a larger dataset that better represents the conditions experienced in this country.

Data Augmentation

Data Augmentation refers to the technique of using existing data to create more data[43]. The idea of data augmentation is that you can create a more diverse dataset which then leads to a more generalized model. I aimed to use Data Augmentation to better represent Irish weather in my training dataset.

The main data augmentation techniques are focused on slight geometric distortions such as translation, rotation, shearing and cropping.



Applying synthetic images to a dataset is another form of image to image translation which I used during my project. I started off by collecting my own pothole training dataset which is described in detail later in the paper. I then used image to image translation models to generate synthetic versions of real pothole images; the only difference is that the synthetic version is under a different condition.

The general idea in deep learning is the more data you have the better the model performs so applying synthetic potholes under different conditions not only creates a more diverse representation of the Irish climate but it also should lead to better performance. Data augmentation is a simple yet useful technique that is used in a variety of machine and deep learning projects.

Data Collection

Data is often seen as the most important part of any machine learning project. High quality real world data leads to better model performance. The following provides a brief explanation of the training, testing and validation datasets used during this project. The data defined here is used for the pothole detection model although parts of the training data was used for image to image translation.

Training data

Training data refers to the data used to train your models. The data collected here is used to teach the object detector I choose how to detect potholes. Training data is particularly important here as the data used here is also used for image generation. For image generation we take a real image of the pothole and then translate it to a synthetic image under a new lighting or weather condition while maintaining the same pothole structure. My initial idea was to fine tune the BDD100k dataset for pothole detection however unfortunately I didn't have the computational resources required to work with such large amounts of data. I settled for the 2019 annotated pothole dataset from Kaggle[44]. A big part of the reason I used this was because TU achieved impressive results in pothole detection using this same dataset in this paper. This was my training dataset meaning my object detector would learn how to detect potholes from this dataset. This dataset includes 618 images with 256 novel images without data augmentation. This dataset includes a variety of different camera distances making it the ideal candidate to train a generalizable model.

Training Data	Kaggle Pothole Data
Acquisition device	Smartphone/ Digital Camera
Image Size	500 x 500 pixels
Lighting Level	Normal: 228 Low: 52
Total Images	256

This dataset took its inspiration from TU Dublin's recent paper which explored how we can train more generalized models[42]. I felt 280 images personally wasn't enough to train a top performing model so I used another dataset as well and combined the images and annotations together into one larger dataset. The 2nd dataset I used is called the pothole detection dataset available on github[45]. It consists of 1,243 annotated pothole images; however I only used 300 images from this dataset. The reason for this was some images were extremely similar to one another, I randomly shuffled the images and picked 300 random images to ensure no similar images were included.

It's worth noting that only the first 256 images collected in the annotated pothole dataset were used for image to image translation later.

Training Data	Pothole detection dataset
Acquisition device	Smartphone/ Digital Camera
Image Size	500 x 500 pixels
Images in dataset	300
Total Images(+ kaggle dat	556

Validation data

Validation data refers to the unseen data first used to test the model. The model's performance on the validation data doesn't define its performance overall. The testing set is generally seen as the more important testing data. Validation data is used so the model can learn its mistakes and gradually improve over new examples. The validation data used during this project was remaining unused images in the YOLO pothole detection dataset described earlier. Generally in machine learning your validation set should include around 15% of your overall dataset. 100 novel images from the pothole detection dataset were used for validation.

Name	Total Images
Pothole detection dataset	100
overall	100

Testing Data

Considering the highly scientific nature of my project the testing data is nearly as important as the training data. The testing data refers to the potholes I use to test my trained object detector on. Since my project is aiming to build a state-of-the-art pothole detector suited to the Irish climate it's only right I use testing data that tests the model under diverse conditions. I ended up choosing two separate testing datasets. The first was the testing data introduced by TU in their paper that explores the performance of pothole detectors under diverse conditions. This dataset provides 3 sub datasets inside each focusing on specific conditions. The first is Cranfield which is then split up into 3 sub datasets based on the distance of the pothole from the camera. The second is Dublin which is split into low light and daylight potholes. And the third is stereo images which looks at how well the model can generalize to different image types. This dataset is extremely diverse making it the ideal candidate for testing.

Name	Image size	Total images
Cranfield	400 x 400	150

Stereo/Non Stereo	400 x 400	75
Dublin	3648 x 2736	80

The second testing dataset was to test the rain in the images. Unfortunately the TU dataset I mentioned includes a wide range of conditions; it doesn't explicitly include images under rainy conditions. To solve this problem I used a second dataset called ‘An annotated water filled and dry potholes dataset’. This dataset provided a desirable mix of potholes under water(rainy) conditions and water under dry conditions. Although the model includes over 500 images we only used 150 images from this dataset. The reason for this is we didn’t want too many of the same style images so we kept a similar amount to the amount used in the other testing datasets. With this now we had 455 images overall for testing. In essence we collected 4 separate datasets and 2 were combined to develop our training data while the other two were combined to form my testing data. Pothole data is generally hard enough to find especially considering the diverse conditions so I was happy with the data I was using. All data for the project was collected through researching existing datasets using sites like paperwithcode and arxiv. Several papers have showed how adverse conditions can negatively affect the performance of object detectors[41]

Irish weather conditions

To create an accurate pothole detection model better suited for the Irish climate than current options the style and conditions of the synthetic data is extremely important. Unfortunately I didn’t have the expertise required to determine what weather conditions I should focus on in my synthetic image generation so I got in contact with Met Eireann for assistance. I get help and recommendations from meteorologist Mark Bowe and scientist Eoin Walsh who both work for Met Eireann. I got in contact initially with the Met Eireann email which then forwarded me onto these two specific men. I have to say both were incredibly helpful with my project. I asked both men the same question ‘if you were to build a pothole detector for Ireland what weather would you ensure was included in the training data’. Mark recommended I ensure that I have Rain, Sun and wind in my dataset. While Eoin said to try and focus on low light dark images. Both men’s inputs allowed me to compare, analyze and eventually decide what conditions I would use.

Mark’s recommendation of Rain was one I was expecting after all Ireland has an awful lot of rain so I was going to generate synthetic rain images. The training dataset I had decided on already included mostly sunny pictures anyway so there was no need for sun pictures and unfortunately wind translation just isn't possible. I couldn't find any pre-trained models for the task or any data I could use to train my own model. I agreed with Eoin’s idea of dark images as as we reach certain times of the year it gets dark quickly, hence its important to have synthetic dark pothole images. I decided I would also add synthetic fog images as highlighted earlier fog appears to decrease object detection accuracy quite a lot. Fog is also a danger to Irish drivers as it does appear from time to time. In the end I chose with the careful assistance of Mark Bowe and Eoin Walsh to add

synthetic rain, fog and low light images to my dataset to create a diverse dataset and train a pothole detector better suited to the Irish climate.

Synthetic Fog Implementation

The goal of the initial data augmentation part of my project is to use my pothole training dataset to generate synthetic images of that data but under a different condition. As already explained three conditions were chosen with the first of those three: fog.

Fog is a regular enough condition here in Ireland. To generate synthetic fog images I had two options to either start from scratch and train my own image to image translator or to implement an existing fog image to image translator. The first option is ideal as you control the architecture however it is hard to find quality fog images and it is highly unlikely that I could train a translator that outperforms the state-of-the-art options available. These state-of-the-art options are typically created by a team of engineers which is the main reason they are so accurate.

Unfortunately as image to image translation is such a new field there are no publicly available comparisons of models, this means I can't just re-implement the best model. My initial plan was to use Cycle GAN and to fine tune it on my data, although I tried this it was highly unsuccessful partially due to the fact that I couldn't find high quality fog images. The reconstruction of the input image was very different from the actual input image which shows that the translation isn't accurate.

Thankfully I came across the 'Foggy CycleGAN' github repo[48]. This featured a state-of-the-art pre-trained model to translate any image into their fog equivalent using Cycle GAN. The model was developed by Ghais Zaher a machine learning engineer, Ghais developed this model for his computer science master thesis for the university of Debrecen. Ghais trains a custom Cycle GAN based on the Cycle GAN principles described earlier to train an image to image translator that can take any input and translate it to the same image with fog in it.

As this repo wasn't developed by me I decided to get in contact with Ghais via email to discuss his work and how it could be applied to my work. Ghais essentially recommended I use his newest weights for the most realistic images and he provided me with the steps to implement his models.

Once I downloaded the pre-trained weights for this model I just had to connect my drive(where the weights were stored) to google colab(the environment i was using to generate the images). After a few blocks of code I was able to translate all my 280 pothole images from the '2019 annotated pothole dataset' into new fog versions using this fine-tuned CycleGAN.

This image shows the real pothole from the annotated pothole dataset.

This image shows the synthesized version of fig 20, cycleGAN applies fog to the image making it trickier to detect the location of some of the potholes.

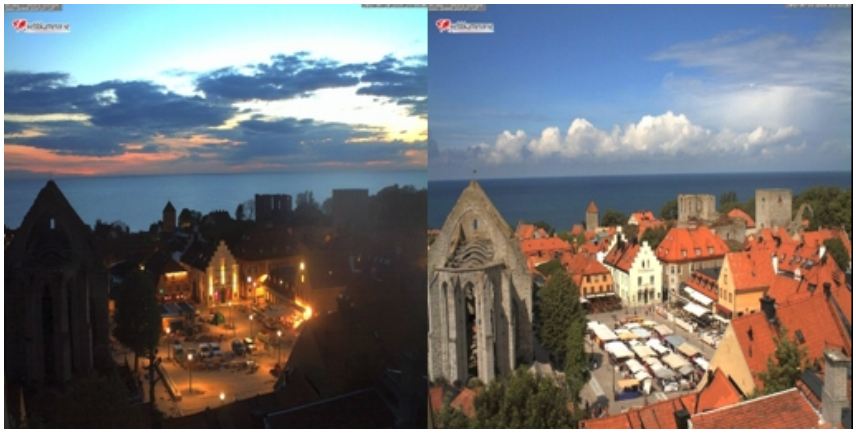
Overall 256 synthetic fog images were generated using the pre-trained foggy Cycle GAN model. The structure of my training dataset could now be described as the following.

Name	Total Images	Real/Fake
Annotated pothole dataset	256	Real
Yolo pothole dataset	300	Real
Synthetic fog images	256	Fake

Synthetic low light implementation

Translating the content of an image from clear or day weather to night weather also sometimes known as low light translation would generally be seen as a task easier than fog translation. The decision was made once again to use a pre-trained model due to their increased performance.

The model chosen is called ‘Day2Night’[49] which is a very large github repo based on image to image translation. This repo provides over 18 pre-trained options to suit the goals of the user. Day2Night unlike Foggy Cycle GAN actually uses UNIT(Unsupervised image to image translation), both are highly developed unsupervised image to image translators that generally share similar performance. The obvious advantage of using pre-trained models for fog and low light translation is I don’t need to manually annotate hundreds of images in both day and night lighting conditions. The idea of pre-trained means the model has already learned the mapping between low light and day light and can perform translation without training from scratch.



The day2night repository actually consists of more than just UNIT models, it also offers cycle GAN and Pix2Pix options however a more diverse set of pre-trained weights were offered for UNIT which was why it was chosen as the forehead of this particular translation.

Day2Night was developed by Dima Goncharenko. This is a project that has lasted years. The models are trained on the BDD100k dataset and some options use VGG(Visual Geometry Group) , a powerful CNN to garner better results. I choose to use the VGG16 UNIT model trained on both the BDD100k and NEXET(largest road dataset in the world) datasets. With these two large datasets there was no need to add any of my data or tune the model.

I cloned the github repo, downloaded the weights and the requirements and then performed the translation is just a few lines of code. Again as it was a pre-trained model the performance was quite impressive.

The real image used for translation from 'an annotated pothole dataset'.

A synthetic version of fig 24. Both images are the exact same expect fig 25 mimics a night-based condition.

256 synthetic night images in total were generated all based on the original 'annotated pothole dataset'. Applying more darkness into my dataset was very important and so did Eoin Walsh, through applying more synthetic images I was now building a larger and more diverse dataset to better represent the Irish climate.

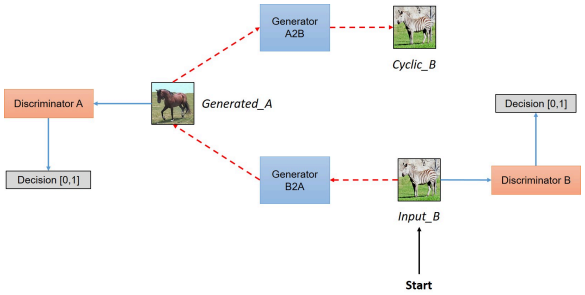
Synthetic Rain images implementation

Translating the content of an image from clear weather to rain would be considered a much trickier task than both fog and low light. For this reason there are no publicly available pre-trained models to translate input images to their rain equivalent. This essentially means I will need to collect my own data to train my own image to image translator to translate clear potholes to rainy potholes.

The process isn't as hard as one might think. I used my second training dataset 'the pothole detection dataset' which was already annotated as my clear weather dataset. While the rain images I collected were from Github labelled 'Rain 100 dataset'. This dataset consisted of 100 images of images under rain conditions so 200 of the clear images were deleted so we would have

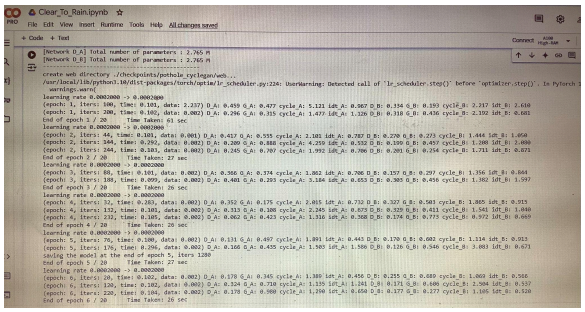
100 clear images and 100 rainy images. With this each clear image would have a corresponding rain image allowing my model to learn the mapping.

The model I selected was CycleGAN as it has a well developed repository and since I had no labels for my rainy images I had to use an unsupervised architecture.



A simplified flowchart of the CycleGAN architecture, it consists of 2 generators and 2 discriminators with cycle consistency the key idea.

I used the Cycle GAN Pytorch implementation respiratory[50]. This provided all the scripts(written in PyTorch)necessary to successfully train and test the model on your own datasets. Google Colab as it had been the full project was used as the editing and code environment. Once the repository was cloned and all my files had been linked to drive and the requirements were downloaded I ran the training script. I chose to train my model over just 60 epochs as I was working with a small dataset meaning it would likely reach its maximum results quick enough.



An image of my google colab setup with the model training one epoch at a time.

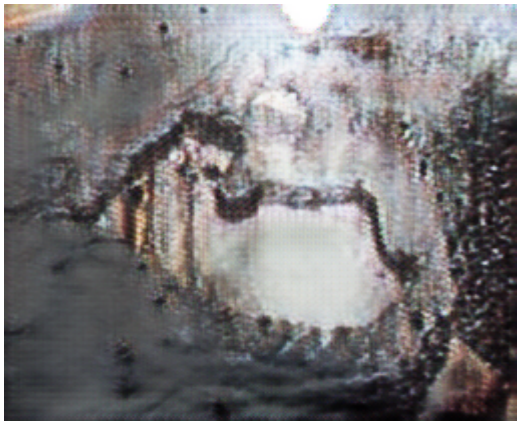
As I use a relatively powerful(also expensive) GPU, the A100 GPU, the model was finished training after around only 20 minutes averaging around 20-25 seconds per epoch. The model was instantly tested on my ‘annotated pothole dataset’. With this I could generate rainy images of my potholes to expand my dataset even more.

This was undoubtedly my least accurate translation however it still provided valuable extra images to my dataset and considering rain is so frequent in Ireland I felt synthetic rain images would be important to leave out if I was to build a pothole detection specifically suited for the Irish climate.

The following shows the results: The reconstruction image is quite different from the input showing slightly less accurate mapping between input and output however overall I am still relatively happy

with the performance. Note: only 74 images were generated as the model disconnected throughout testing.

The input image for the rain translation



The output image after translation, darkness and rain are added although it is difficult to see.

The reconstruction of the input image through the second generator. Overall its very accurate the lighting is just slightly different.

New Datasets

Overall we generated 586 synthetic images through a variety of conditions. These conditions were conditions suited to Ireland's climate. The hope is that by using this augmented dataset we can outperform a model that just uses real images and doesn't include synthetic images.

With the new images it's worth noting that their annotations already actually exist in the dataset used as the input data so there was no need to manually label them all. I did have to manually organize my drive to set up for my experiments. 3 main datasets were gathered for the final experiment which is described in detail later in the paper. The following tables break down each dataset.

Name	Total images	Real/Fake
Annotated pothole dataset	256	Real
Pothole detection dataset.	272	Real
Combined(both datasets together.	528	Real

This table breaks down my real dataset, this dataset acts as the baseline dataset during our experiments.

Name	Total Images	Real/Fake
Fog Potholes	256	Fake
Dark Pothole	256	Fake
Rain Pothole	74	Fake
Combined	586	Fake

Breakdown of synthetic dataset. This dataset only includes images generated by image to image translators and is completely fake. Note duplicate images are there as the same pothole is there under different conditions which could lead to decreased performance.

Name	Total Images	Real/Fake
Annotated pothole data	256	Real
Pothole detection dataset	272	Fake
Fog pothole	100	Fake
Dark pothole	100	Fale
Rain pothole	74	Fake

Comhined	802	Hybrid

Breakdown of our hybrid dataset, my hypothesis expects this model to perform best. We only choose 100 for the synthetic images so synthetic images would still be the majority in the dataset.

It's worth noting that the first 100 images were selected for the foggy synthetic images and the next 100 hundred were selected for the dark synthetic images with the 74 rain synthetic images. This was done so there were very little duplicates in the dataset. This should lead to slightly better results.

Deep learning model

With the first part of the project now completed in generating synthetic images it was now up to the second part. This part of the project is where the pothole detection actually happens. As stated before I am trying to train a model that offers unique advantages to the Irish climate so it is important to choose a model that offers good performance in adverse weather conditions. Object detection was chosen as the computer vision method of detecting potholes due to its simplicity yet impressive results. Object detection is most usually what method the state-of-the-art pothole detectors use.

Object Detection choice

Earlier this paper I analyzed the best performing state-of-the-art object detection models. I looked at the developments in each model and how they build on each other, both single shot and double shot detectors were analyzed.

Papers were analyzed and the general conclusion found was that the YOLO architecture was the best suited model to object detection under adverse conditions. What's great about this is that YOLO is a super fast object detector meaning nearly all its sizes can run at real time speed. Generally YOLO has been seen as the best object detector for several years now with EfficientDet and SSD also highly recommended.

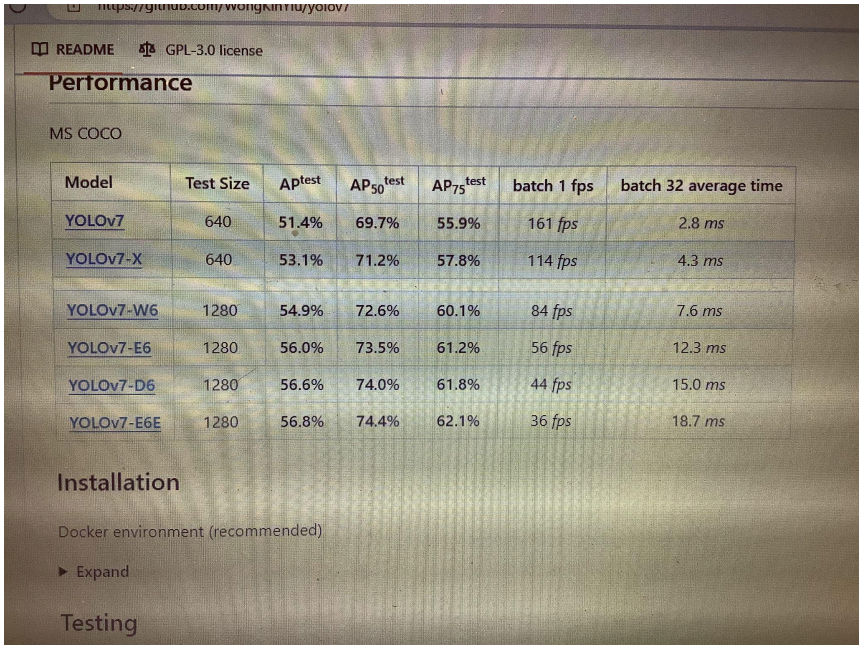
As mentioned previously YOLO is a large family of object detection models with 11 different versions in total being developed to date. YOLOv11 is the newest version of the YOLO family[47], all different models slightly built on top of the model before it and offer new features.

YOLOv11 the newest model supposedly offers the best performance however YOLOv8 is the most developed. I planned on testing all versions and comparing them however I unfortunately didn't have the computational resources to do this(My A100 GPU would not cover 11 different models not to mention there are 5-6 different sizes for each version). For that reason I had to rely on existing research to decide which version to use. This paper tests all the top recent object detectors under adverse conditions. The paper shows that YOLOv7 offers the best performance with YOLOv8 close. I made the decision to use YOLOv7 as my model[46]

YOLO v7 size experiment

YOLOv7 offers 6 different sizes. The size refers to how many layers the architecture uses. The general idea would be the bigger the size the more accurate the model is but is slightly slower. It's important to reach at least 30 FPS if I want my model to run in real time so the decision was made to the choose prior to experimentation that accuracy was the priority for the project as long as it was still above 30 FPS. 30 FPS is generally seen as the slowest possible model considered fast enough for real time in object detection.

The YOLOv7 sizes are below.


README GPL-3.0 license

Performance

MS COCO

Model	Test Size	AP^{test}	AP_{50}^{test}	AP_{75}^{test}	batch 1 fps	batch 32 average time
YOLOv7	640	51.4%	69.7%	55.9%	161 *fps*	2.8 *ms*
YOLOv7-X	640	53.1%	71.2%	57.8%	114 *fps*	4.3 *ms*
YOLOv7-W6	1280	54.9%	72.6%	60.1%	84 *fps*	7.6 *ms*
YOLOv7-E6	1280	56.0%	73.5%	61.2%	56 *fps*	12.3 *ms*
YOLOv7-D6	1280	56.6%	74.0%	61.8%	44 *fps*	15.0 *ms*
YOLOv7-E6E	1280	56.8%	74.4%	62.1%	36 *fps*	18.7 *ms*

Installation

Docker environment (recommended)

▶ Expand

Testing

As mentioned previously generally the slower the model is the more accurate it is however this is not always the case with object detection. My experiments described in detail later showed this. Prior to experimentation the decision was made to use the YOLOv7 -D6 version. This version is the second most powerful and also the second slowest. The reason this model was selected was it has nearly identical accuracy with the most powerful model in YOLO v7 E6E while being a good bit faster. The YOLO v7 D6 model uses a high image resolution and includes more layers than the other sizes in the YOLOv7 family.

I used the pothole detection dataset from YOLO as my testing data for this particular task as it provided a simple dataset to evaluate the different sizes on. All sizes were then validated on a small fraction of the pothole detection dataset.

My experiment essentially tested all 6 sizes of the YOLO v7 family and compared two key metrics mAP 0.5 and speed. Results show later in the paper that YOLO v7 D6 is actually not only faster

than YOLO V7 E6E but it is also more accurate. YOLOv7-d6 and YOLOv7-e6e both score in the top ten for the COCO object detection leaderboard[53]

Transfer Learning

Transfer learning refers to using the knowledge gained to solve one problem and then apply it to another. Human's use transfer learning every day for tasks[51]. Transfer learning is very popular in machine learning as an extremely useful technique when working with small amounts of data.

The key idea with transfer learning is that by training one powerful model on a large dataset to detect something, the knowledge gained from solving that problem can be used to solve a related problem. For example a model that learns how to detect cars could apply to detecting trucks. Transfer learning has shown to improve accuracy on object detection tasks especially when working with a small dataset like I am.

Training

The format YOLOv7 and most other YOLO versions use is that the input data images must be a jpg image while each jpg image must have a corresponding text annotation file containing bounding box coordinates for each object. All my images were sorted into this format for training, validation and testing.

YOLO also requires a data.yaml file be written. This file gives YOLO the files to your training and validation datasets(and testing but after the model is trained) as well as the number of objects and the name for the objects. I defined two objects: a pothole and a background image. The background image is the same as no pothole.

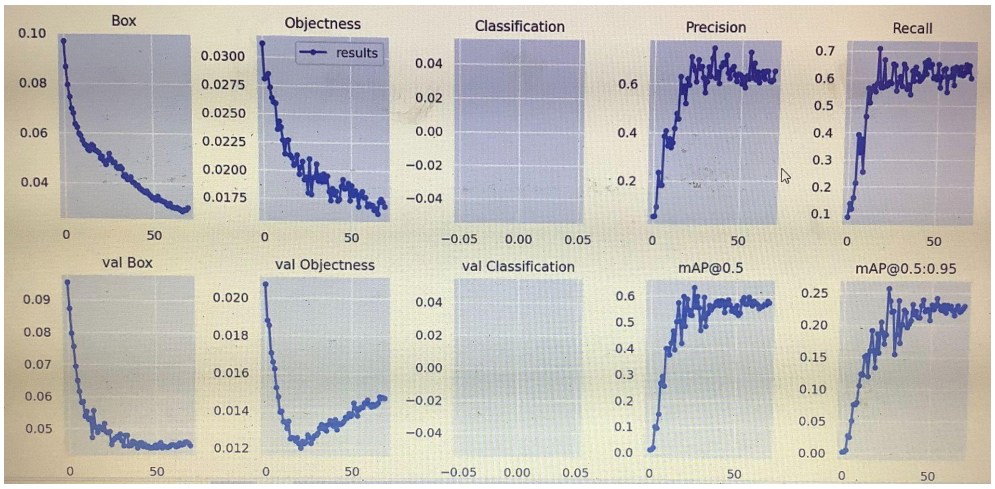
As mentioned previously I used transfer learning for training my YOLO V7 D6 model. YOLO is trained on a well known objects dataset called the COCO dataset. This dataset consists of a staggering 328, 000 images of labelled objects. YOLO doesn't learn pothole detection through this dataset but it does learn an extremely strong mapping between different objects and different weather and lighting conditions making it a valuable addition to my model.

This experiment was designed as follows: Three different datasets would be used to train a YOLOV7 D6 model using transfer learning. Each model would be trained on the same number of epochs and same image resolution size. The three datasets being trained are the aforementioned Real, Synthetic and Hybrid datasets. Each model will be tested on the testing data described earlier in the paper with 5 key evaluation metrics being recorded. Those being precision, recall, mAP 0.5, mAP 0.5-0.95 and inference speed. The goal is to compare the strengths and weaknesses of each specific model and evaluate the impact synthetic images has. Before experimentation my hypothesis was and still is ‘through applying synthetic images mimicking common irish conditions we can train a more accurate and suited object detection model to Ireland’s diverse climate.

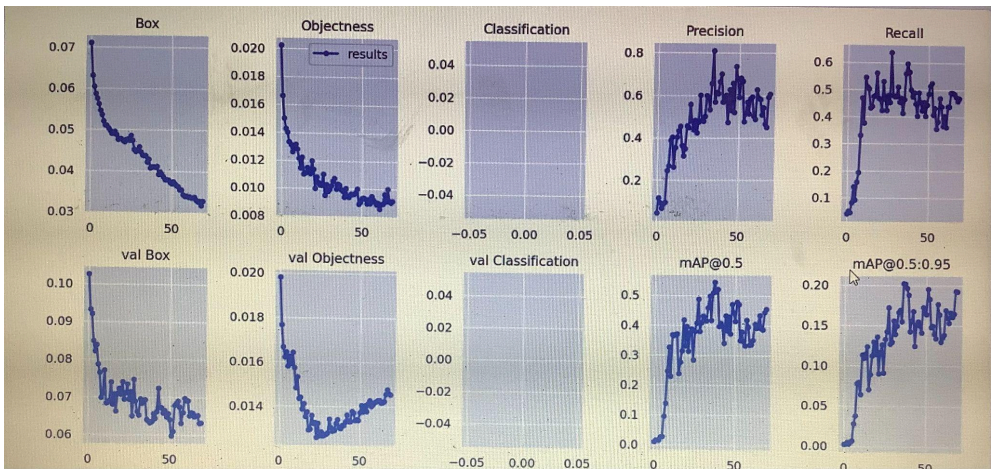
All training was conducted on a Google Colabatory Pro account using an A100 GPU. Due to the highly experimental nature of my project over 30 models were trained with different image resolution, different number of epochs, different parameters and most importantly different data.(This was solely for experimentation to work out the optimal parameters and sizes)

Each of the three described dataset were trained using Python and the YOLO v7D6 architecture. Each of the three main models were trained over 100 epochs at an image size of 820 x 820 image resolution. The learning rate was the standard learning rate while after hours of training on end we had each model trained.

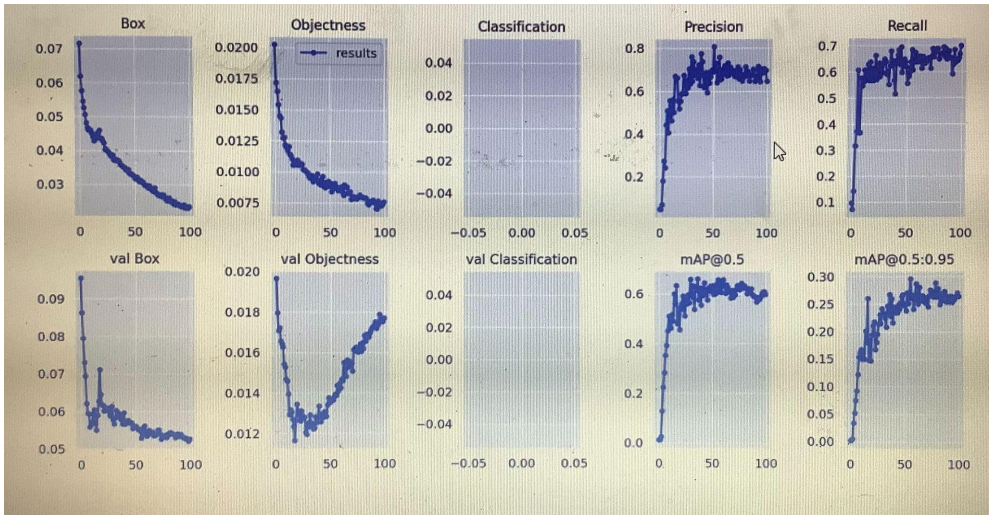
Below is a comparison of the training results of each model. Note: it’s the testing results that are important; these just show how capable each model is on the validation set but not the testing set which is discussed later in the paper.



Key evaluation metrics from the YOLOv7-D6 trained on real images only.



Key evaluation metrics from the YOLOv7 -D6 trained solely on synthetic images



Key evaluation metrics from YOLOv7-D6 trained on a hybrid dataset consisting of both real and synthetic images.

These 3 models were the main three models after much experimentation before, YOLO v7-D6 proved very capable of pothole detection with the mAP 0.5 and mAP 0.95 generally increasing over each epoch.

Results

The following section presents the results I achieved from my testing.

Experiments

Two main experiments took place. The first was testing each YOLO v7 size variant to identify which model was fastest and which model was most accurate. This most accurate model would then be used to train the pothole detector. The second experiment is more important. This experiment trains 3 separate models using the 3 different datasets described above. The results of this experiment will reveal the impact of synthetic images on improving object detection accuracy and whether or not it is a viable approach.

Evaluation metrics

Evaluation metrics are used to assess and compare the performance on your machine learning models with existing models. There are many different object detection evaluation metrics that can be used but the most important metrics that were kept track of during my experimentation was precision, recall, mAP(mean average precision) 0.5 and inference speed[52].

Recall

Recall is a popular evaluation metric used in object detection. Recall is the metric used to identify the object detection model's ability to detect objects in an image. Recall examines if the model can identify all objects in an image. Assuming there are 10 potholes inside an image and the object detection model identifies 9 of the 10 potholes, its recall is 90%.

The recall formula is described as following:

$$Recall = true\ positives / true\ positives + false\ negatives$$

True positives mean any object that was detected while false negatives refer to the objects missed or not detected.

Precision

Precision is another popular evaluation metric used in object detection. Precision is calculated after recall and it determines how accurate the predictions made are. If the recall says there are 8 objects even though there are 10, precision is used to calculate how many of these 8 predictions have the right object detected(object class). Assuming 6 of the model's object predictions are correct then the precision is 75%.

The precision formula is described as following:

$$Precision = true\ positives/true\ positives + false\ positives$$

False positives refer to predicted objects that are predicted as the incorrect class.

mAP 0.5

mAP stands for mean average precision is the most comprehensive metric in object detection. It calculates the average precision for all recall values and then works out the mean across every class. mAP is essentially the metric that shows how well the model's precision and recall is overall. The 0.5 at the end means that any object that has an intersection over union of 0.5 meaning that 0.5 or more of the object is detected is considered successful while anything less is incorrect.

Speed

Speed can be measured in various ways but the simplest method is measuring the amount of time it takes to process images. Most object detectors calculate this themselves meaning no calculations are required for the developer. Speed is important as if a model isn't fast enough it isn't going to be able to run at real time.

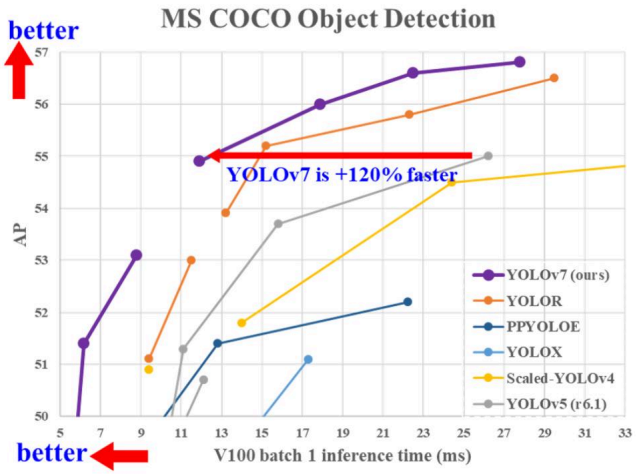
Results

This section displayed the results obtained through rigorous experimentation.

YOLO v7 size results

The YOLOv7 family is made up of 6 different sizes, all offering unique advantages over each other. As mentioned before YOLO is an extremely efficient and quick object detector meaning it aligns with my goals of being deployable at real time. YOLOv7 is currently ranked 7th on the paperswithcode real time object detection on the COCO dataset leaderboard[53]. Only YOLOv6

provides better real time performance of the YOLO family. One key difference between YOLOv1 which was described earlier and the YOLOv7 model is that YOLOv7 implements compound scaling just like EfficientDet, this leads to not only more accuracy but more efficiency too.



A performance graph comparing YOLOv7 to previous models. YOLOv7 shows a dramatic speed improvement.

All 6 sizes were tested during this experiment. The 6 sizes were trained on the ‘annotated pothole’ dataset and tested on the ‘pothole detection dataset’ these two datasets are actually used for training the final models but for the sake of testing which YOLOv7 size works best with potholes they were split up again here for training and testing. The testing dataset includes a nice mix of potholes allowing the test to be beneficial. All 6 models were trained with an image size of 820 x 820 pixels and were trained over 75 epochs. The results can be summarized with the following table:

Model	Map 0.5	ms/image
YOLO v7	0.778	**4.9 ms**
YOLO v7X	0.779	6.2ms
YOLO v7-w6	0.566	8.0 ms
YOLO v7-e6	0.512	9.4ms
YOLO v7-d6	**0.819**	7.0 ms
YOLOv7 e6e	0.782	7.4ms

A table breaking down the performance of the different YOLO size variants. It’s evident some models were not suited well at all to pothole detection.

These results draw some very interesting conclusions. One is that YOLO models or at least YOLOv7 wide(w6) and extended(e6) models are not suited to pothole detection. Both models demonstrate extremely poor performance in both accuracy and speed. This could be due to a variety of reasons. Another interesting conclusion is that YOLOv7’s models aren't organized from

least accurate to most accurate or fastest to slowest like most YOLO models are. YOLO v7 has more unique architectures with the e and d models adding extensions to the baseline model but not always improvements.

The 2 most impressive models were the YOLO v7 baseline model and the YOLO v7-d6 model. Both performed very well overall. I decided to use the YOLO v7 d6 model as it offered nearly a 3 percent accuracy increase. The tradeoff was losing just over 2 milliseconds processing speed. The YOLO v7-d6 model was ultimately chosen for its superior accuracy while still being very fast.

Baseline vs Synthetic vs Hybrid

This experiment included training 3 YOLO v7-d6 models on the same parameters over the same number of epochs. A comprehensive testing set was used to examine different aspects of each model. The testing data included:

- Cranfield images - these images were used to test how well the model could detect potholes from different distances from the camera.
- Dublin road images - these images were used to experiment how well the model could detect Irish potholes taken under tricky lighting conditions.
- Stereo images - this dataset was used to examine how well the model could perform when tested on a new type of image stereo images. Considering my model wasn't trained on stereo images it tests the models adaptability.
- Water and dry dataset - this dataset was used to test how well the model could perform when tested on wet and damp conditions similar to the conditions we experience in Ireland.
- All data - all 455 images from the 4 datasets described above were combined to test each model overall ability.

Baseline results

This model was trained on 564 real potholes. No synthetic images were included.

Dataset	images	precision	recall	mAP 0.5
Water and dry dataset	150	0.788	0.64	0.65

Baseline results table 1: overall poor enough performance.

dataset	images	precision	recall	Map 0.5
300 cranfield	50	0.944	0.962	0.978
400 cranfield	50	0.98	0.943	0.972
500 cranfield	50	0.999	0.943	0.97
All cranfield	150	0.974	0.943	0.97

Baseline Table 2: A breakdown of the performance of the different cranfield datasets and them all combined on our baseline dataset model. Performance overall is near immaculate.

dataset	images	precision	recall	Map 0.5
Dublin Low-Light	40	0.65	0.734	0.673
Dublin Day-Light	40	0.722	0.634	0.643
Dublin All	80	0.641	0.719	0.663

BaselineTable 3: A breakdown of the results of Dublin potholes in day and low light as well as the combined results. Overall average performance as these are tricky potholes to detect.

dataset	images	precision	recall	Map 0.5
Medium stereo	25	0.76	0.76	0.64
Close non stereo	25	0.883	0.6	0.737
Medium non stereo	25	0.911	0.82	0.899
All non	50	0.881	0.74	0.833

stereo				

Baseline Table 4: A table to break down the results of the baseline model when tested on a new kind of images in stereo images and non stereo images from different distances - note there was a close stereo file but its images were too small to be used.

Dataset	images	precision	recall	Map 0.5
All data above combined	455	0.845	0.71	0.763

Baseline Table 5: A table that breaks down the entire testing set on the baseline non synthetic YOLOv7-d6 model. Overall a map 0.5 of 76% percent means the baseline approach yield very solid accuracy.

One evident conclusion from this work is that the model's struggles badly with the annotated pothole water dataset, the model also struggles with stereo images and dublin images.

Synthetic results

This model was trained on the exact same parameters, image resolution and number of epochs that all other models were trained on; the only difference was that this model was trained on 586 synthetic AI generated images. No real pothole was used in the training data.

are the following:

Dataset	images	precision	recall	mAP 0.5
Water and dry dataset	150	0.757	0.714	0.715

Synthetic results table 1: .An evident improvement in performance by just using synthetic images in detecting pothole under wet conditions.

dataset	images	precision	recall	Map 0.5
300 cranfield	50	0.81	0.887	0.791
400 cranfield	50	0.889	0.904	0.879

500 cranfield	50	0.854	0.886	0.848
All cranfield	150	0.847	0.874	0.832

Synthetic Table 2: An evident decrease in accuracy. In comparison to the baseline non synthetic model this model loses about 10% of its mAP accuracy which is disappointing.

dataset	images	precision	recall	Map 0.5
Dublin Low-Light	40	0.619	0.663	0.518
Dublin Day-Light	40	0.647	0.536	0.494
Dublin All	80	0.644	0.573	0.498

Synthetic Table 3: When tested in Dublin images synthetic images actually provided the worst results. The precision was similar however the recall and mAP 0.5 lose considerable accuracy.

dataset	images	precision	recall	Map 0.5
medium stereo	25	0.92	0.92	0.929
Close non stereo	25	0.437	0.56	0.514
Medium non stereo	25	0.686	1	0.868
All non stereo	50	0.573	0.779	0.696

Synthetic Table 4: Mixed results while synthetic images bring a massive increase in mAP 0.5 to stereo images it has a different impact on non-stereo with non-stereo close performing worse.

Dataset	images	precision	recall	Map 0.5

All data above combined	455	0.757	0.714	0.715

Synthetic Table 5: Overall when tested on all data the synthetic only image approach offers a slight recall improvement but as expected is a little short of the baseline approach which has outperformed it.

Although overall the baseline approach has outperformed the synthetic approach, the synthetic approach offers certain unique improvements those most evident include a 3.4 percent mAP 0.5 increase when tested on wet and damp data which is very topical to Ireland's climate, we also see an an even more impressive 28 percent mAP 0.5 improvement when the model is tested with stereo images. Considering the model isn't actually tested on stereo images this showcases that synthetic images can actually adapt better to new image kinds than real pothole images which is very interesting.

My idea was why not use the best of both worlds and use a hybrid model trained on both synthetic and real potholes. With this not only do we have 802 images more than any other model we also get a great representation of conditions that are topical to the Irish climate.

Hybrid results

This model was trained on the exact same number of parameters as the other models however for this model we used a hybrid approach consisting of 802 pothole images. 274 of these images were synthetic while the rest of the potholes were real images. This dataset was easily the most diverse dataset with multiple different weather conditions included in the dataset.

The hybrid results model was trained on 802 images consisting of the entire baseline dataset as well as 100 synthetic fog images, 100 synthetic low light images and 74 synthetic rain images. We didn’t use all data to avoid duplicates although based on the synthetic results that may not have been as much of an issue as I anticipated. The model was trained on 100 epochs with the same parameters as the other two models and the results are as follows.

Dataset	images	precision	recall	mAP 0.5
Water and dry dataset	150	0.767	0.684	0.736

Hybrid results table 1: A nice improvement over the synthetic results and a significant improvement over the baseline results tested on this data, all round the best results to date on wet/damp images.

dataset	images	precision	recall	Map 0.5
300 cranfield	50	0.923	0.906	0.945
400 cranfield	50	0.927	0.962	0.982
500 cranfield	50	0.927	0.962	0.984
All cranfield	150	0.925	0.937	0.97

Hybrid Table 2: It's evident that non synthetic potholes are more effective for Cranfield images with a significant improvement in performance here compared to synthetic, it's similar to performance to the baseline with Cranfield data with very high accuracy.

dataset	images	precision	recall	Map 0.5
Dublin Low-Light	40	0.666	0.732	0.614
Dublin Day-Light	40	0.68	0.731	0.732
Dublin All	80	0.719	0.718	0.65

Hybrid Table 3: Just like the annotated water dataset the hybrid approach garners the best results with some nice improvements when working with Dublin road potholes. That being said Dublin still has the lowest results by quite a bit compared to the other sets.

dataset	images	precision	recall	Map 0.5
medium stereo	25	1	0.92	0.953
Close non stereo	25	0.732	0.88	0.824
Medium non stereo	25	0.695	0.875	0.837

All non stereo	50	0.711	0.8	0.806

Hybrid Table 4: Overall easily the best approach, the stereo images Map is considerably improved through using synthetic images while non-stereo images still have a good performance.

Dataset	images	precision	recall	Map 0.5
All data above combined	455	0.779	0.774	0.809

Hybrid Table 5: over a 4% mAP 0.5 increase in comparison to the next best model shows that the hybrid approach is undoubtedly the best option.

This was all done through rigorous experimentation and testing new parameters, image sizes and epochs. Overall it is very evident from the tables shown that a Hybrid based approach that uses both synthetic and real pothole images is the best way to train pothole detection models.

All models average 7.0ms per image meaning they have an FPS(framer per second) of 142 which is considered ultra fast and efficient.

Discussion

After a bit of comparison between the different models some results stand out to me in particular. These results showcase the potential of using synthetic images not only in pothole detection projects but also in any project related to object detection.
The difference between the baseline model and the hybrid model when tested on stereo images was a staggering 31.3% percent mAP 0.5 increase. The hybrid model also achieved perfect precision scoring 1. This substantial increase showcases that synthetic images don't only make your model better in performing in adverse conditions but they also show the use of synthetic images to train a model that can adapt to new image types better.

Another staggering accuracy difference between the hybrid model and the baseline model was their respective mAP 0.5 on the annotated water and dry dataset. The hybrid approach showed an 8.6% percent mAP 0.5 increase as well as a 12.7% percent increase in recall. The model was tested here under adverse conditions in damp and rainy potholes and the baseline model struggled. With the help of synthetic images however mimicking the weather conditions it is tested on we can drastically increase the accuracy of detection even under adverse conditions.

The Cranfield results were very similar all around and the hybrid model also brought improvements to the Non Stereo images and Dublin Images. It is worth noting that Dublin images was the obvious

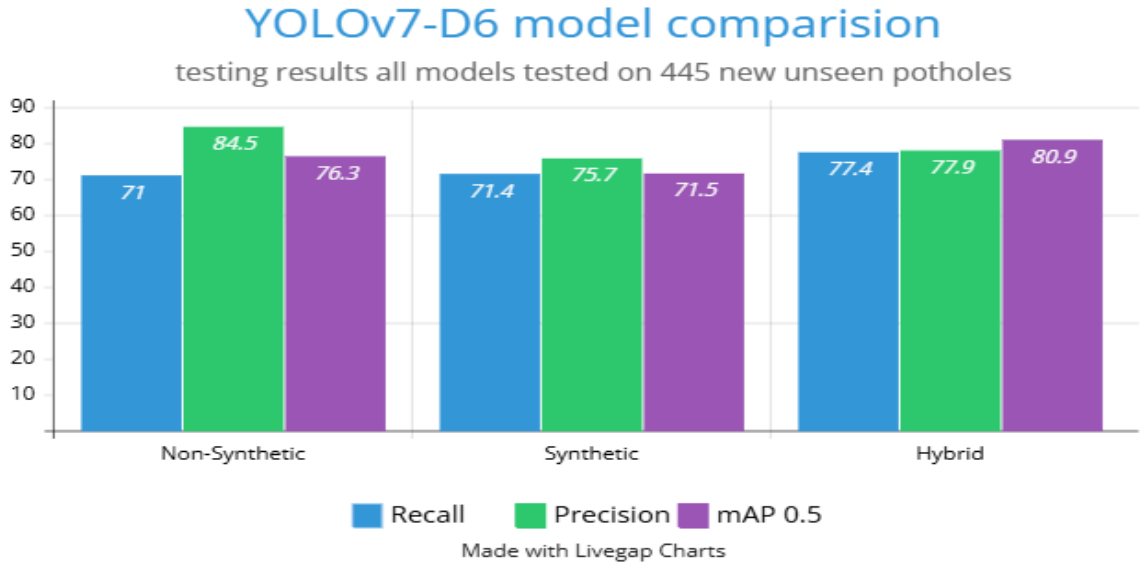
outlier in this experiment, even with synthetic data the model still struggles to reach 0.7 mAP 0.5. This suggests new methods and ideas must be applied to solve this problem.

Overall we see 4.6% percent increase in the hybrid models mAP 0.5 in comparison to the baseline model when tested on all data. A 4% increase is a big increase and I was very happy with this, even just a 4% increase can lead to significant improvement in terms of safety for the Irish public when using Irish roads.

Overall my hypothesis worked out and synthetic images did prove very capable. Synthetic images brought some interesting improvements and overall improved the detection of potholes under adverse conditions. I believe my model is better than current options as it prioritizes the Irish climate with its training data, shows considerable improvements under wet and damp testing data and is also more adaptable than options that dont use synthetic data.

Although FPS wasn't measured per model we used the YOLOv7 D6 architecture which claims from its paper that it runs at 44 fps. This is considered very fast and is definitely fast enough to be deployed at scale.

Future work includes training with a larger dataset for even more accurate results. Another future experiment will be to test different proportions of synthetic images, I used about 70% non synthetic data and 30% synthetic data to build my hybrid model, although this was successful, different combinations could yet yield even better results.



Conclusion

The main goal of my work was to train a pothole detection model that can be used as an aid for manual road inspectors. With this model manual road inspectors can simply take a picture of hundreds, even thousands of potholes and upload them to my model which will then output the bounding boxes for each pothole detected. My model used synthetic data generated from unpaired image to image translation models to create a more diverse representation of the climates we experience here in Ireland. My results clearly show that a hybrid based training approach is the best approach. The hybrid model which incorporated real and synthetic images of potholes outperformed every other method tested in nearly every single way. My hybrid model not only outperforms non-synthetic models under adverse weather conditions such as rain and fog but it also is far more adaptable than non synthetic models. In the end I developed a more generalized pothole detection system that limits bias through using synthetic images. These synthetic images were designed to represent some common adverse conditions experienced here in Ireland and through rigorous experimentation I have built my own novel pothole detector. I believe my project has massive applications to improve the safety of Irish citizens and also greatly reduce the expenditure of the government and country councils spent on manual pothole inspection. My model showed fantastic results when tested on a variety of different lighting conditions, distance from camera to pothole and different pothole sizes. This more generalized model holds huge potential to revolutionize the Irish road industry and make travelling a safer and more enjoyable experience for all citizens.

Real world applications

Although the purpose of my project was to develop a more generalized pothole detector for the Irish authorities and government my project can be expanded much further.

Here are some areas why my project may include advantages over existing approaches.

- Pothole detection in third world countries: With road conditions worse than ever in third world countries my project was designed to not only be accurate but also fast meaning it can be deployed without major costs. My pothole detector provides a simple option to help third world countries solve or at least improve their pothole issues. My hybrid model includes extremely diverse data meaning it could achieve a solid performance on nearly any climate.
- Airport runways: Airport runways have seen issues with potholes before and this can put passengers and the flight crew at risk. My model provides a simple, scalable and accurate solution for Irish or International airports to detect potholes quickly and at a low price.

My project can be employed in various situations, not just pothole detection in Ireland. Due to it’s diverse dataset, improved ability to adapt to new image types and increased accuracy under diverse conditions my model(particularly my hybrid model) provides an ideal solution for many different issues faced regarding road maintenance. My model processes images at a speed of

7.0ms per image meaning it runs at 142 FPS which is an extremely fast time. This means there is a relatively little cost to use my model at scale.

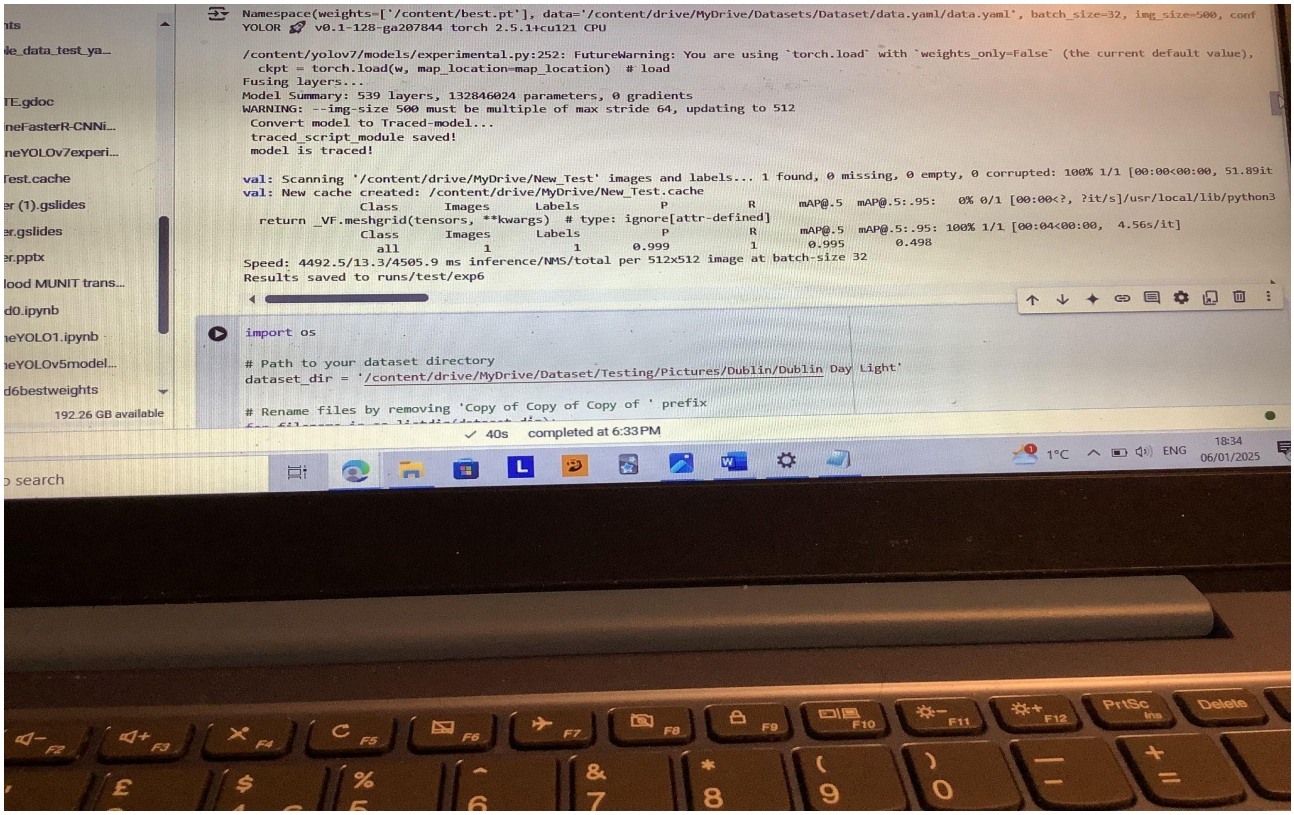
Appendices

Appendix A - Real world pothole detection

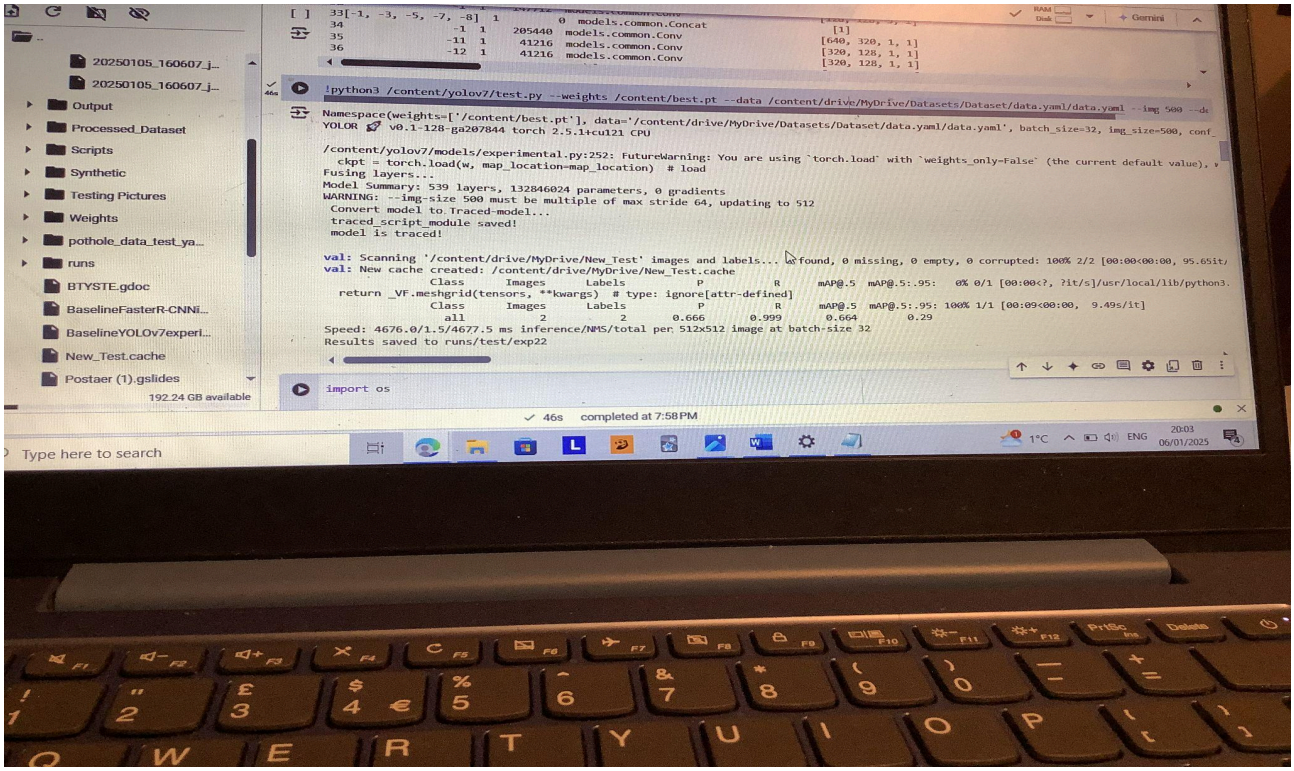
As real world detection was seen as a key goal project I went outside with my phone and took 3 pictures of potholes on my own road. It was a damp day so the potholes were all relatively wet and damp. Roboflow was used to annotate each image while each image was tested individually using my most accurate model in YOLO v7-d6 hybrid.

The images used were:

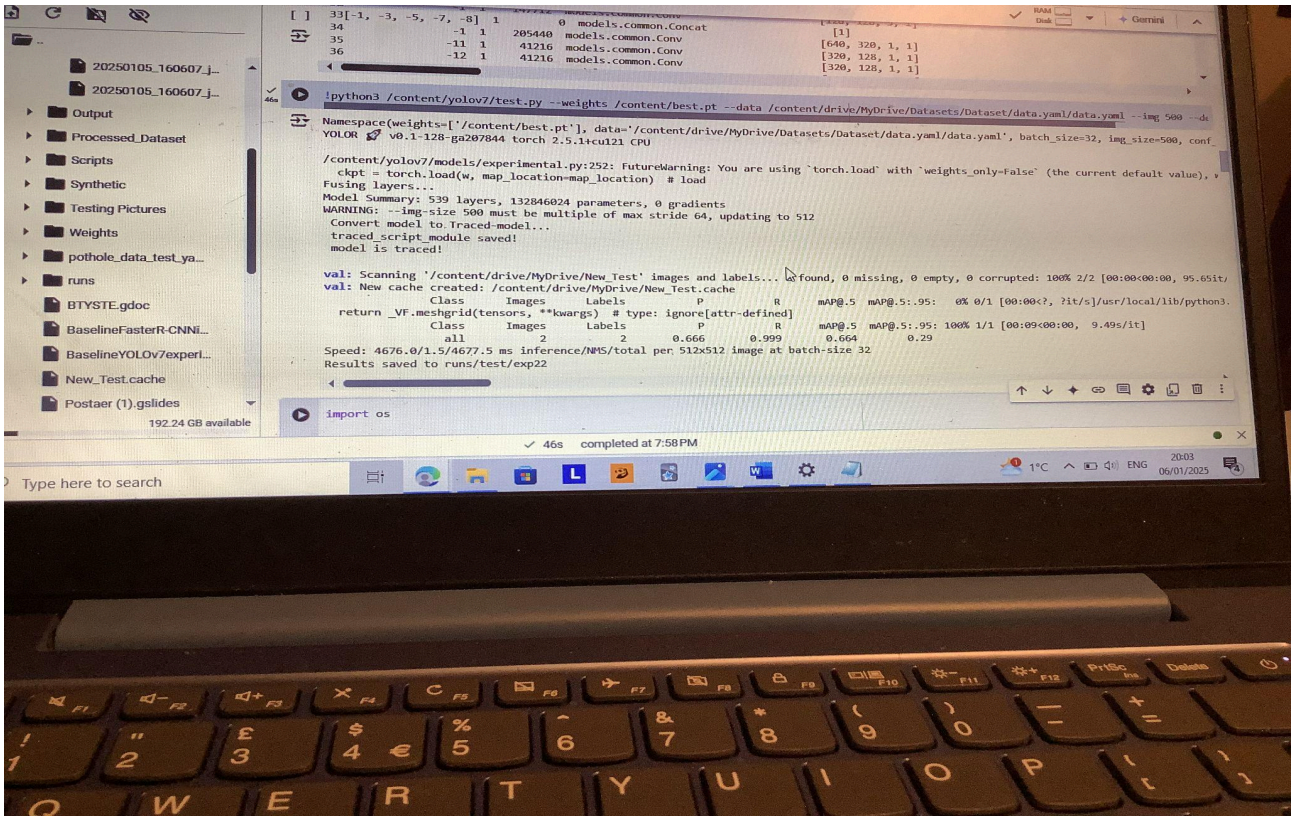
One pothole was present here and my model detected it with a 99.5% mAP 0.5 indicating perfect accuracy.



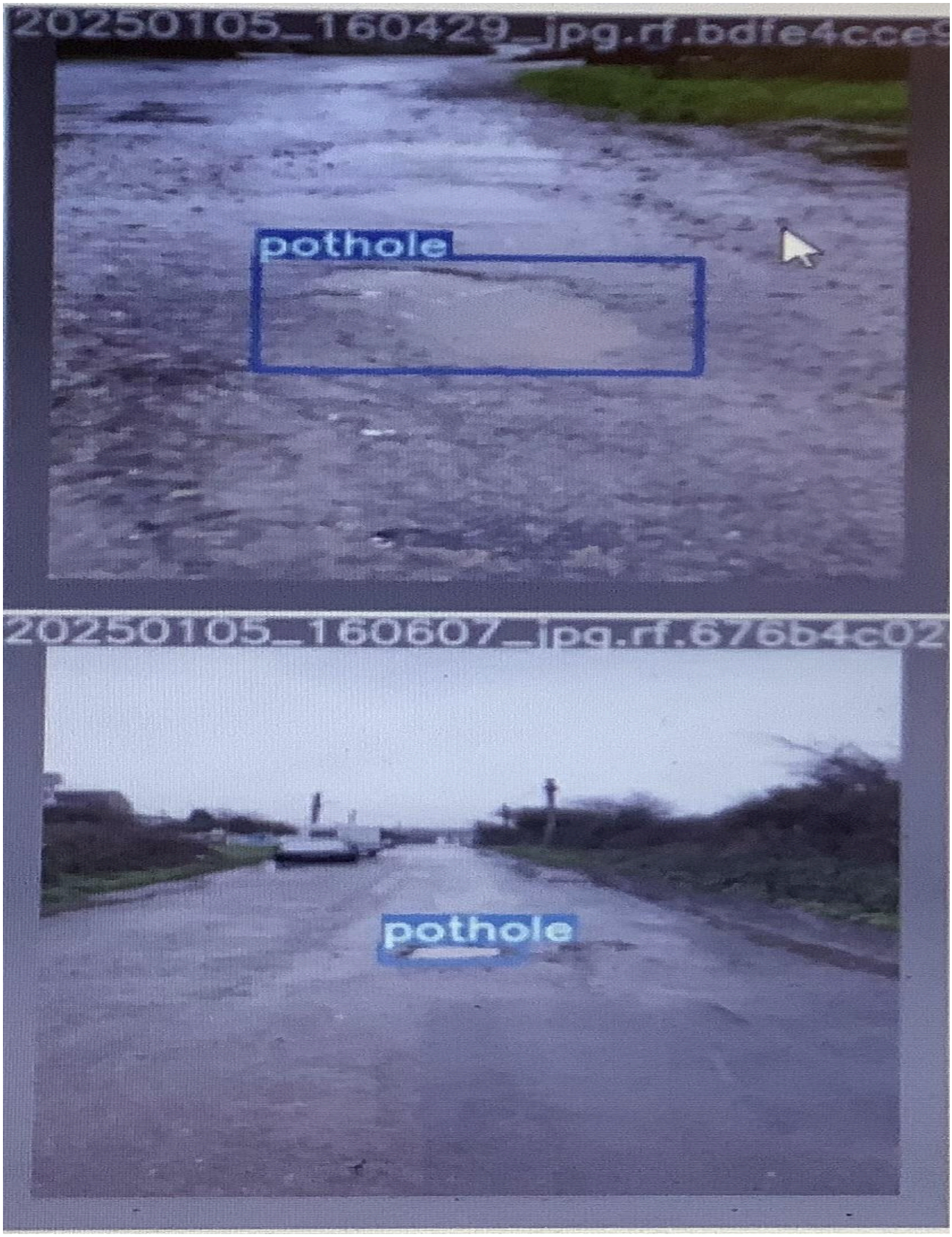
This second image included 2 potholes 1 being obvious while the other being extremely difficult to detect. As expected the performance did drop scoring a 66.4 mAP 0.5



This last image was another challenging image. 2 clear potholes are at the front of the image although they are minor while one large pothole is also there. The model scored a 69 mAP 0.5 which it was impressed with as it is a tricky image.



This image below shows the bounding box coordinate for 2 of the images.



Appendix B - YOLOv7-d6 code

The following is the code used to implement transfer learning and train my YOLO v7 -d6 hybrid model - best performing model.

```
# clone repository
!git clone https://github.com/WongKinYiu/yolov7.git
!cd yolov7
```

```
#Download requirements
!pip install -r /content/yolov7/requirements.txt
```

```
# download model weights
!wget
https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-d6.pt
```

```
# mount drive
from google.colab import drive
drive.mount('/content/drive')
```

```
# move weights into main repository
!mv yolov7-d6.pt /content/drive/MyDrive/Dataset

# training script with parameters
!python3 train_aux.py --workers 8 --device 0 --batch-size 8 --data
/content/drive/MyDrive/Dataset/data.yaml/data.yaml --img 832 832 --cfg
cfg/training/yolov7-e6e.yaml --weights /content/yolov7/yolov7-e6e.pt --name
yolov7e6e-custom --epochs 100
```

```
# testing script
!python3 /content/yolov7/test.py --weights /content/best.pt --data
/content/drive/MyDrive/Datasets/Dataset/data.yaml/data.yaml --img 500
--device 'cpu'
```

The code for the synthetic and non synthetic models is nearly identical. Google Colab was adapted as the code editor for each model.

The code for training cycleGAN for image to image translation.
Rain translation:

```
# clone the repository being used
!git clone https://github.com/junyanz/pytorch-CycleGAN-and-pix2pix.git
!cd pytorch-CycleGAN-and-pix2pix
```

```
# install requirements
!ls /content/pytorch-CycleGAN-and-pix2pix/requirements.txt
!pip install -r /content/pytorch-CycleGAN-and-pix2pix/requirements.txt
```

```
#  mount drive to access data from my drive for training
from google.colab import drive
drive.mount('/content/drive')
```

```
# set up to train model
!python train.py --dataroot /content/drive/MyDrive/Datasets --name
pothole_cyclegan --model cycle_gan
```

```
# train model
!python /content/pytorch-CycleGAN-and-pix2pix/train.py --dataroot
/content/drive/MyDrive/Datasets --name pothole_cyclegan --model cycle_gan
--display_id -1 --n_epochs 60 --n_epochs_decay 10
```

```
Test model to translate my images to rain
!python /content/pytorch-CycleGAN-and-pix2pix/test.py --dataroot
/content/drive/MyDrive/Datasets --name pothole_cyclegan --model cycle_gan
--epoch latest
```

Low light translation:

```
# Download model weights
!wget --no-check-certificate -r
"https://docs.google.com/uc?export=download&id=1mrj0vDzuFufpmxSW5SMIAn9Xekeg
X4Hh" -O code.zip
!unzip -o code.zip -d /content/day2night
```

```
# mount drive and download requirements
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
!pip install /content/requirements.txt
```

```
# downloading vgg weights to add to existing weights - leads to more
powerful models
import torch
from torchvision import models
vgg = models.vgg16(pretrained=True)
torch.save(vgg.state_dict(),
"/content/day2night/UNIT/checkpoints/vgg/vgg/vgg16.pth")
```

```
# translate images
!python3 /content/day2night/UNIT/test_batch.py --device cpu --config
"/content/drive/MyDrive/Weights/Copy of config.yaml" --input_folder
"/content/drive/MyDrive/Dataset/Images/Pictures" --output_folder
"/content/drive/MyDrive/Datasets/BTYSTE/Output-Images" --checkpoint
"/content/drive/MyDrive/Weights/gen_01000000.pt" --a2b 1 --trainer UNIT
--recon
```

Low light translation(note this code is copied from the repo not novel code)

```
# connect colab
import sys
import os

colab = 'google.colab' in sys.modules
    os.environ['PROJECT_DIR'] = project_dir = '/content/Foggy-CycleGAN'
    replace = True
    if os.path.isdir(project_dir):
        choice = input("Project already exists in folder " +
                       "{}\nDelete the files and pull again? Enter Y/(N):\n"
                       .format(project_dir))
        if choice.lower() == 'y':
            !rm -r $PROJECT_DIR
            print("Deleted folder {}".format(project_dir))
        else:
            replace = False
            print("Nothing was changed.")
    if replace:
        !cd /content && git clone
https://github.com/ghaiszaher/Foggy-CycleGAN.git
        print("Project cloned to " + project_dir)
    os.chdir(project_dir)
    print("Done.")
```

```
om lib.tools import create_dir

output = './weights/generator_clear2fog.weights.h5'
if not os.path.exists(output):
    import gdown
    file_id = '10cBLNuofYGbyrC6DKRycD-fmx0Wh7BCr'
    create_dir('./weights/')
    print("Downloading weights...")
    gdown.download(f'https://drive.google.com/uc?id={file_id}', output,
quiet=False)
    print("Weights downloaded.")
else:
    print("Weights already downloaded.")
```

```
from lib.models import ModelsBuilder

models_builder = ModelsBuilder()
generator_clear2fog = models_builder.build_generator()
generator_clear2fog.load_weights('./weights/generator_clear2fog.weights.h5')
```

```
from lib.plot import plot_clear2fog_intensity
import tensorflow as tf
from lib.dataset import DatasetInitializer

datasetInit = DatasetInitializer(256, 256)

input_path = './custom-images/input/image.png'  #@param {type:"string"}
fog_intensity = 0.4  #@param {type:"slider", min:0.05, max:1.00, step:0.05}
output_path = './custom-images/output/image.png'  #@param {type:"string"}

image_clear = tf.io.decode_png(tf.io.read_file(input_path), channels=3)
image_clear, _ = datasetInit.preprocess_image_test(image_clear, 0)

fig, foggy_image = plot_clear2fog_intensity(generator_clear2fog,
image_clear, fog_intensity)
foggy_image_uint8 = tf.image.convert_image_dtype(foggy_image,
dtype=tf.uint8)
encoded_image = tf.io.encode_png(foggy_image_uint8)
tf.io.write_file(output_path, encoded_image)
print(f"Saved foggy image to {output_path}")
```

Appendix C - Synthetic Images

Here are 5 real pothole images and their 3 translation images in fog, rain and low light conditions. All were generated using unpaired image to image translation models. Although 586 synthetic images in total were generated I have simplified it to 15 for this appendix to not make this report book too long. The images are organised from real(non synthetic), fog(synthetic), low light(synthetic) to rain(synthetic). Note: the fog images are slightly less as they were changed to another size by default.

Image 1

Image 2

Image 3

Image 4

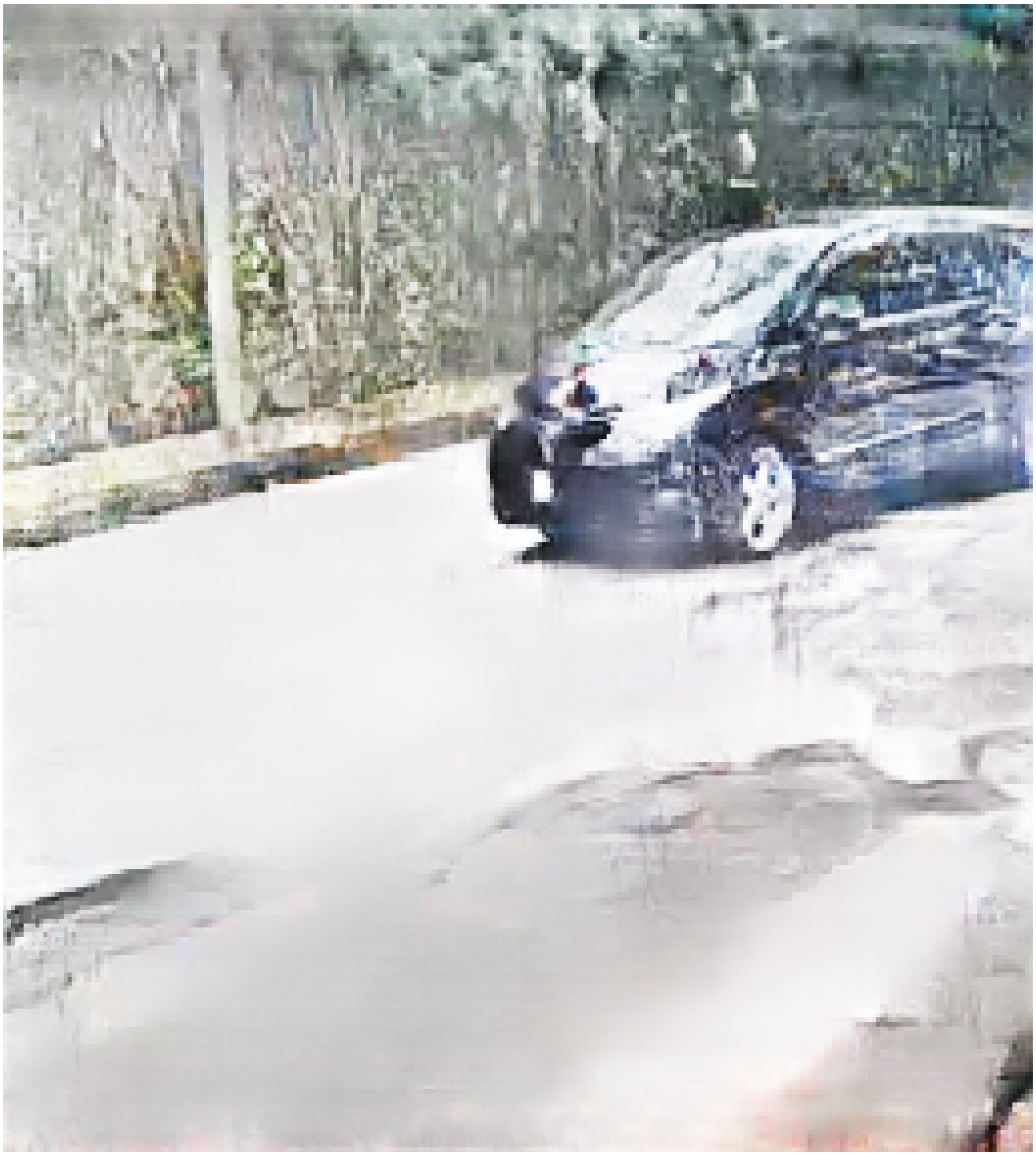


Image 5

References

[1] ‘Majority of Irish drivers have had vehicles damaged by potholes, research finds’ - “The Irish Examiner - https://www.irishexaminer.com/news/arid-30985000.html
[2] ‘NI roads: Potholes still affecting rural journeys after £500m spend’ - “BBC https://www.bbc.com/news/articles/c2dlzxl52yno
[3] ‘’Staggering cost of fixing potholes last year revealed as fed-up Brits report a MILLION road craters - “The Irish Sun’ https://www.thesun.ie/motors/13020679/potholes-cost-councils-56million-repairs/
[4] ‘ Artificial Intelligence and Bias: Challenges, Implications, and Remedies’ -” ResearchGate https://www.researchgate.net/publication/374493754_Artifical_Intelligence_and_Bias_Challenges_Implications_and_Remedies
[5] ‘Risk and the future of AI: Algorithmic bias, data colonialism, and marginalization’ - ScienceDirect - “ScienceDirect https://www.sciencedirect.com/science/article/abs/pii/S1471772723000325
[6] Improved YOLOv9-Based Objects Detection in Adverse Weather Conditions for Autonomous driving - ScienceDirect - “ScienceDirect https://www.sciencedirect.com/science/article/pii/S2405896324022961
[7] ‘Object Detection in Adverse Weather for Autonomous Driving through Data Merging and YOLOv8’ - “PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC10611033/
[8] ‘Transport Environmental Indicators Ireland 2022 - Central Statistics Office’ - “TFI https://www.cso.ie/en/releasesandpublications/ep/p-eii/environmentalindicatorsireland2022/transport
[9] ‘How Do Potholes Form: Understanding the Causes and Prevention | McConnell & Associates’ - “Mconnell associates How Do Potholes Form: Understanding the Causes and Prevention | McConnell & Associates
[10] The Hidden Costs and Inefficiencies of Manual Road Inspections - “Linkedin https://www.linkedin.com/pulse/hidden-costs-inefficiencies-manual-road-inspections-roadvisionai-lumzc/
[11] ‘City of san diego asking residents to report potholes’ - “7 San Diego https://www.nbcsandiego.com/news/local/city-of-san-diego-asking-residents-to-report-potholes/
[12] ‘Christchurch the pothole capital of New Zealand’ - https://www.neighbourly.co.nz/public/christchurch/christchurch-central/message/39740441
[13] ‘Vehicle driving assistant’ - paperswithcode https://paperswithcode.com/paper/vehicle-driving-assistant
[14] ‘Arthur Samuel’ - “Wikipedia https://en.wikipedia.org/wiki/Arthur_Samuel_(computer_scientist)
[15] Etham Alpaydin ‘Introduction_to_Machine_Learning_Ethem_Alpaydin’ - Physical book
[16] ‘Supervised Machine Learning’ - GeeksforGeeks https://www.geeksforgeeks.org/supervised-machine-learning/
[17] ‘Unsupervised Learning’ - Cambridge ul.pdf
[18] ‘Data Labeling: An Empirical Investigation into Industrial Challenges and Mitigation Strategies’ - ResearchGate https://www.researchgate.net/publication/347079938_Data_Labeling_An_Empirical_Investigation_into_Industrial_Challenges_and_Mitigation_Strategies
[19] Geoff Hinton ‘Learning representations by back-propagating errors’ - Nature https://www.nature.com/articles/323533a0#:~:text=We%20describe%20a%20new%20learning%20procedure%2C%20back-propagation%2C%20for,of%20the%20net%20and%20the%20desired%20output%20vector
[20] Fukushima, K.. "Visual feature extraction by a multilayered network of analog threshold elements". IEEE Transactions on Systems Science and Cybernetics. 5 (4): 322–333. doi:10.1109/TSSC.1969.300225
[21] Yann LeCun ‘Handwritten recognition with a Back-Propagation Network’ - NeurIPS https://proceedings.neurips.cc/paper/1989/file/53c3bce66e43be4f209556518c2fcb54-Paper.pdf
[22] ‘Computer Vision for Road Imaging and Pothole Detection: A State-of-the-Art Review of Systems and Algorithms’ - paperswithcode https://paperswithcode.com/paper/computer-vision-for-road-imaging-and-pothole
[23] ‘ Multi-Scale Context Aggregation by Dilated Convolutions’ - Arxiv https://arxiv.org/abs/1511.07122

[24] ‘Deep Learning-based Pothole Detection for Intelligent Transportation: A YOLOv5 Approach’ The sai https://thesai.org/Downloads/Volume14No12/Paper_42-Deep_Learning_based_Pothole_Detection_for_Intelligent_Transportation.pdf#:~:text=The%20proposed%20approach%20leverages%20dilated%20convolutional%20neural%20networks,and%20global%20contextual%20information%2C%20enhancing%20the%20detection%20performance.
[25] Mingxing Tan, Quoc v Le ‘EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks’ - Arxiv https://arxiv.org/abs/1905.11946
[26] Ian Goodfellow ‘Generative Adversarial Networks’ - IEEE Xplore https://ieeexplore.ieee.org/document/9966215
[27] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun ‘Deep Residual Learning for Image Recognition’ - Arxiv https://arxiv.org/abs/1512.03385
[28] Ilya Sutskever, Oriol Vinyals, Quoc V. Le, ‘Sequence to Sequence Learning with Neural Networks’ - Arxiv https://arxiv.org/abs/1409.3215
[29] Kyle Aitken, Vinay V Ramasesh, Yuan Cao, Niru Maheswaranathan, ‘Understanding How Encoder-Decoder Architectures Attend’ - Arxiv https://arxiv.org/abs/2110.15253
[30] Yingxue Pang, Jianxin Lin, Tao Qin, Zhibo Chen, ‘Image-to-Image Translation: Methods and Applications’ - Arxiv https://arxiv.org/abs/2101.08629
[31] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, Alexei A. Efros, ‘Image-to-Image Translation with Conditional Adversarial Networks’ - Arxiv https://arxiv.org/abs/1611.07004
[32] Jun-Yan Zhu, Richard Zhang, Deepak Pathak, Trevor Darrell, Alexei A. Efros, Oliver Wang, Eli Shechtman, ‘Toward Multimodal Image-to-Image Translation’ - Arxiv https://arxiv.org/abs/1711.11586
[33] Jun-Yan Zhu, Taesung Park, Phillip Isola, Alexei A. Efros, ‘Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks’ - Arxiv https://arxiv.org/abs/1703.10593
[34] Ming-Yu Liu, Thomas Breuel, Jan Kautz, ‘Unsupervised Image-to-Image Translation Networks ’ - Arxiv https://arxiv.org/abs/1703.00848
[35] Ross Girshick, Jeff Donahue, Trevor Darrell, Jitendra Malik, ‘Rich feature hierarchies for accurate object detection and semantic segmentation’ - Arxiv https://arxiv.org/abs/1311.2524
[36] Ross Girshick, ‘Fast R-CNN’ - Arxiv https://arxiv.org/abs/1504.08083
[37] Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun, ‘Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks - Arxiv https://arxiv.org/abs/1506.01497
[38] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi, ‘You Only Look Once: Unified, Real-Time Object Detection’ - Arxiv https://arxiv.org/abs/1506.02640
[39] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, Alexander C. Berg, ‘SSD: Single Shot MultiBox Detector’ - Arxiv https://arxiv.org/abs/1512.02325
[40] Mingxing Tan, Ruoming Pang, Quoc V. Le, ‘EfficientDet: Scalable and Efficient Object Detection’ - Arxiv https://arxiv.org/abs/1911.09070#:~:text=In%20this%20paper%2C%20we%20systematically%20study%20neural%20network,and%20propose%20several%20key%20optimizations%20to%20improve%20efficiency.
[41] ‘Performance Evaluation of Object Detection Algorithms Under Adverse Weather Conditions’ - eudl https://eudl.eu/pdf/10.1007/978-3-030-71454-3_13
[42] Syed Hassan, Dymphna O Sullivan, Dr Susan Mckeever, ‘Pothole detection under diverse conditions using object detection models’ - ResearchGate - https://www.researchgate.net/publication/351294430_Pothole_Detection_under_Diverse_Conditions_using_Object_Detection_Models
[43] ‘Data Augmentation’ - Wikipedia https://en.wikipedia.org/wiki/Data_augmentation
[44] ‘Annotated pothole images dataset’ - Kaggle https://www.kaggle.com/datasets/chitholian/annotated-potholes-dataset
[45] ‘Pothole detection dataset’ - Github https://github.com/jaygala24/pothole-detection

[46] ‘Comparison of CNN-Based Models for Pothole Detection in Real-World Adverse Conditions: Overview and Evaluation’ Semantic Scholar https://pdfs.semanticscholar.org/0ebd/d4398f524c625a3b33859486cb2468112d1e.pdf
[47] ‘YOLOv11’ - Ultraylytics https://docs.ultralytics.com/models/yolo11/
[48] ‘Foggy CycleGAN’ - Github https://github.com/ghaiszaher/Foggy-CycleGAN
[49] ‘Day2Night’ - Github https://github.com/solesensei/day2night
[50] ‘CycleGAN and Pix2Pix’ - Github https://github.com/junyanz/pytorch-CycleGAN-and-pix2pix
[51] ‘What is Transfer Learning? - GeeksforGeeksWhat is Transfer Learning? - GeeksforGeeks’ - GeeksforGeeks https://www.geeksforgeeks.org/ml-introduction-to-transfer-learning/
[52] ‘Evaluating object detection models’ - GeeksforGeeks https://www.geeksforgeeks.org/evaluating-object-detection-models-methods-and-metrics/
[53] ‘Object detection leaderboard’, paperswithcode https://paperswithcode.com/sota/real-time-object-detection-on-coco?p=yolov7-trainable-bag-of-freebies-sets-new

Acknowledgements

I would like to acknowledge the following people for their help, recommendations and support during this project.

- I would like to acknowledge professor Andrew Hines from UCD for his support and recommendations.
- I would like to thank Syad Hassan from TU Dublin for guiding me to the right datasets to use and where to find them.
- I would like to thank Eoin Walsh and Mark Bowe from Met Eireann for their contribution towards my selection of which weather conditions to generate.
- I would like to thank 2021 BTYSE winner Greg Tarr for his recommendations and advice.
- I would like to thank my parents for their continuous support.
- I would like to thank BT for hosting this event.